

计算机组成原理对比王道辅导书补充

第一章

1.1 计算机系统层次结构

对比王道 1.2.4 计算机系统的层次结构

（一）未解释微体系结构的实现方式

微体系结构由逻辑电路（logic circuit）实现，当然，微架构中的一个功能部件可以用不同的逻辑来实现，用不同的逻辑实现方式得到的性能和成本有差异。最后，每个基本的逻辑电路都是按照特定的器件技术（device technology）实现的。

（二）未解释微体系结构和 ISA 的区别

ISA 是软件和硬件之间的接口，是软件可感知的部分，而微体系结构是 ISA 具体的实现组织，是软件不可感知的部分。

【1800 题举例】

18. 关于计算机系统层次结构的叙述，下列错误的是（ ）。

- I. 汇编语言和机器语言都是机器级语言
 - II. 对于微程序机器层来说，它是一个实在的硬件层面，由微程序解释机器指令
 - III. ISA 属于硬件和软件的接口定义，但其不能被软件可见
 - IV. 对于高级语言层，其通过编译程序翻译成汇编语言程序
- A. I、II B. II、III C. I、III D. III、IV

个人补充：

属于 ISA 规定的内容：

①指令格式、指令操作类型和指令的操作数类型；②程序可访问的寄存器的数量、位数和编号；③寻址方式和可访问的地址空间大小；④I/O 空间的编址方式和数据的大、小端存储；⑤机器工作状态的定义和切换；⑥中断结构和存储保护模式；⑦输入输出结构和数据传送方式。

属于微体系结构规定的内容：

①流水线的级数和深度；②超标量发射的宽度；③Cache 的级数和映射方式④ALU 的数量和运算速度；⑤分支预测器的类型和大小；⑥总线的宽度和频率；⑦指令执行的时钟周期数；⑧控制单元的实现；⑨数据通路的组织。

【1800 题举例】

1.下列关于指令集体系结构（ISA）描述正确的是（ ）。

- A. ISA 完整定义了软件和硬件之间的接口，但没有规定计算机该执行什么操作
- B. ISA 规定了操作数的类型，操作数的寻址方式，但没有规定大小端的存储方式
- C. ISA 规定了程序可访问的寄存器编号，个数和位数以及存储空间的大小与编址方式
- D. ISA 规定了执行过程的控制方式，但不包括程序计数器、条件码的定义

2.【2022 年统考真题】下列选项中，属于指令集体系结构（ISA）规定的内容是（ ）。

- | | | | |
|-----------------|---------------|-----------|--------------|
| I. 指令字格式和指令类型 | II. CPU 的时钟周期 | | |
| III. 通用寄存器个数和位数 | IV. 加法器的进位方式 | | |
| A. 仅I、II | B. 仅I、III | C. 仅II、IV | D. 仅I、III、IV |

3.下列关于指令集体系结构（ISA）中指令格式的说法正确的是（ ）。

- A. 变长指令格式能更有效地利用存储空间，因为所有指令长度都根据其功能进行了优化，不会有空间浪费；定长指令格式则会因指令长度固定而产生大量空间浪费
- B. 定长指令格式的优势在于硬件设计简单，指令译码速度快，因为指令长度相同，硬件可以采用统一的译码方式；而变长指令格式需要更复杂的译码电路来处理不同长度的指令
- C. 无论是定长还是变长指令格式，都不会对指令集的功能产生影响，只是在存储和译码方式上有所不同
- D. 定长指令格式比变长指令格式更适合复杂指令集（CISC），因为 CISC 中的复杂指令需要固定长度来保证准确性

4.指令集体系结构（ISA）定义了计算机系统的多个方面，以下（ ）不属于 ISA 的定义范畴。

- A. 指令格式
 - B. 寄存器组织
 - C. 微程序控制器的实现细节
 - D. 寻址方式
5. 【2025 年统考真题】下列选项中，由指令集体系结构（ISA）规定的是（ ）。
- A. 是否采用阵列乘法器
 - B. 是否采用定长指令字格式
 - C. 是否采用微程序控制器
 - D. 是否采用单总线数据通路式

对比王道 1.2.5 计算机系统的不同用户

（一）未解释透明的定义

在计算机技术中，一个存在的事物或概念从某个角度看似乎不存在，即感觉不到实际存在的事物或概念，则称其是透明的。通常，在一个计算机系统中，系统程序员所看到的底层机器级的概念性结构和功能特性对高级语言程序员(通常就是应用程序员)来说是透明的，即看不见或感觉不到。

【1800 题举例】

1. 在 CPU 的寄存器中，（ ）对用户是完全透明的。
- A. 程序计数器
 - B. 指令寄存器
 - C. 状态寄存器
 - D. 通用寄存器
2. 【2010 年统考真题】下列寄存器中，汇编语言程序员可见的是（ ）。
- A. 存储器地址寄存器（MAR）
 - B. 程序计数器（PC）
 - C. 存储器数据寄存器（MDR）
 - D. 指令寄存器（IR）
3. 【2021 年统考真题】下列存储器中，汇编语言程序员可见的是（ ）。
- I. 指令寄存器
 - II. 微指令寄存器
 - III. 基址寄存器
 - IV. 标志状态寄存器
- A. 仅I、II
 - B. 仅I、IV
 - C. 仅II、IV
 - D. 仅III、IV

对比王道 1.2.6 计算机系统的工作原理

（一）缺少程序与指令及控制信号的关系的说明

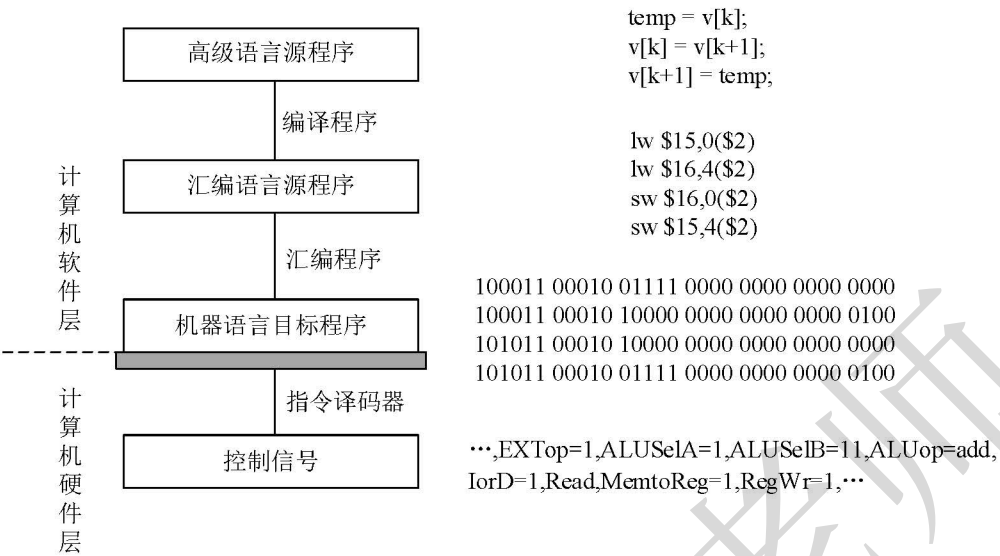


图 1.11 不同层次语言之间的等价转换

图 1.11 给出了实现两个相邻数组元素交换功能的不同层次语言的描述。如图 1.11 所示，在高级语言源程序中，可直观地用三条赋值语句实现两个相邻数组元素的交换。在经编译生成的汇编语言源程序中，上述三条赋值语句可转换为 4 条汇编指令，其中，两条是取数指令 `lw(load word)`，另两条是存数指令 `sw(store word)`。在经汇编后生成的机器语言目标程序中，对应的机器指令是特定格式的二进制代码，例如，汇编指令“`lw $15,0($2)`”的机器代码为“100011 00010 01111 0000 0000 0000 0000”，这是一条 MIPS 体系结构中的指令，其中，高 6 位“100011”为操作码，随后 5 位“00010”为寄存器编号 2，再后面 5 位“01111”为寄存器编号 15，最后 16 位为立即数 0。

CPU 能够通过逻辑电路直接执行用二进制表示的机器指令。指令执行时，通过控制器中的指令译码器将指令操作码进行译码，以解释成控制信号（control signal）来控制数据通路的执行。例如，图 1.11 中给出的控制信号中，`ALUOp=add` 控制 ALU 进行加操作；`Read` 控制进行主存读操作；`RegWr=1` 控制将结果写入寄存器。

整个关系可以总结为：

以相邻数组元素交换功能为例，高级语言的 3 条赋值语句经编译生成 4 条 MIPS 汇编指令，再经汇编转换为二进制机器指令；CPU 通过指令译码器将机器指令操作码译码为控制信号，进而控制数据通路完成指令执行。

第二章

2.2 运算方法和运算部件

对比王道 2.2.1 基本运算部件

(一) 未解释并行进位加法器的逻辑表达式实现

全加器用来实现两个本位数加上低位进位生成一位本位和以及一位向高位的进位。第 i 位的加法运算是指第 i 位的加数 X_i 、 Y_i ，和低位来的进位 C_{i-1} 三者相加，得到本位和 F_i ，和第 i 位的进位输出 C_i 。

F_i 和 C_i 的逻辑表达式如下：

$$\left. \begin{aligned} F_i &= X_i \oplus Y_i \oplus C_{i-1} \\ C_i &= X_i C_{i-1} + Y_i C_{i-1} + X_i Y_i \end{aligned} \right\} \quad (3-1)$$

由全加器公式 (3-1) 可知，对于一个 4 位加法器，其进位 C_1 、 C_2 、 C_3 和 C_4 的产生条件为：

$$\begin{aligned} C_1 &= X_1 Y_1 + (X_1 + Y_1) C_0 \\ C_2 &= X_2 Y_2 + (X_2 + Y_2) C_1 \\ &= X_2 Y_2 + (X_2 + Y_2) X_1 Y_1 + (X_2 + Y_2) (X_1 + Y_1) C_0 \\ C_3 &= X_3 Y_3 + (X_3 + Y_3) C_2 \\ &= X_3 Y_3 + (X_3 + Y_3) [X_2 Y_2 + (X_2 + Y_2) X_1 Y_1 + (X_2 + Y_2) (X_1 + Y_1) C_0] \\ &= X_3 Y_3 + (X_3 + Y_3) X_2 Y_2 + (X_3 + Y_3) (X_2 + Y_2) X_1 Y_1 + \\ &\quad (X_3 + Y_3) (X_2 + Y_2) (X_1 + Y_1) C_0 \\ C_4 &= X_4 Y_4 + (X_4 + Y_4) C_3 \\ &= X_4 Y_4 + (X_4 + Y_4) [X_3 Y_3 + (X_3 + Y_3) X_2 Y_2 + (X_3 + Y_3) (X_2 + Y_2) X_1 Y_1 + \\ &\quad (X_3 + Y_3) (X_2 + Y_2) (X_1 + Y_1) C_0] \\ &= X_4 Y_4 + (X_4 + Y_4) X_3 Y_3 + (X_4 + Y_4) (X_3 + Y_3) X_2 Y_2 + (X_4 + Y_4) (X_3 + Y_3) \\ &\quad (X_2 + Y_2) X_1 Y_1 + (X_4 + Y_4) (X_3 + Y_3) (X_2 + Y_2) (X_1 + Y_1) C_0 \end{aligned}$$

从以上公式来看，每个进位表达式中都含有 $(X_i + Y_i)$ 和 $X_i Y_i$ ，所以，定义两个辅助函数如下：

$$\left. \begin{aligned} P_i &= X_i + Y_i \\ G_i &= X_i Y_i \end{aligned} \right\} \quad (3-2)$$

P_i 称为进位传递函数，其含义是：当 X_i 、 Y_i 中有一个为1时，若有低位进位输入，则一定被传递到高位。这个进位可看作低位进位越过本位直接向高位传递。 G_i 称为进位生成函数，其含义是：当 X_i 、 Y_i 均为1时，不管有无低位进位输入，本位一定向高位产生进位输出。

将 P_i 、 G_i 代入前面 $C_1\sim C_4$ 式中，可得：

$$\left. \begin{aligned} C_1 &= G_1 + P_1 C_0 \\ C_2 &= G_2 + P_2 G_1 + P_2 P_1 C_0 \\ C_3 &= G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 C_0 \\ C_4 &= G_4 + P_4 G_3 + P_4 P_3 G_2 + P_4 P_3 P_2 G_1 + P_4 P_3 P_2 P_1 C_0 \end{aligned} \right\} \quad (3-3)$$

从上述表达式可以看出， C_i 仅与 X_i 、 Y_i 和 C_0 有关，相互间的进位没有依赖关系。只要 $X_1\sim X_4$ 、 $Y_1\sim Y_4$ 和 C_0 同时到达，就可几乎同时形成 $C_1\sim C_4$ ，并同时生成各位的和。

实现上述逻辑表达式(3-3)的电路称为先行进位部件(carry lookahead unit, CLU)。通过这种进位方式实现的加法器称为全先行进位加法器(carry lookahead adder, CLA)。因为各个进位是并行产生的，所以是一种并行进位加法器。图3.5为4位CLU和4位CLA示意图。

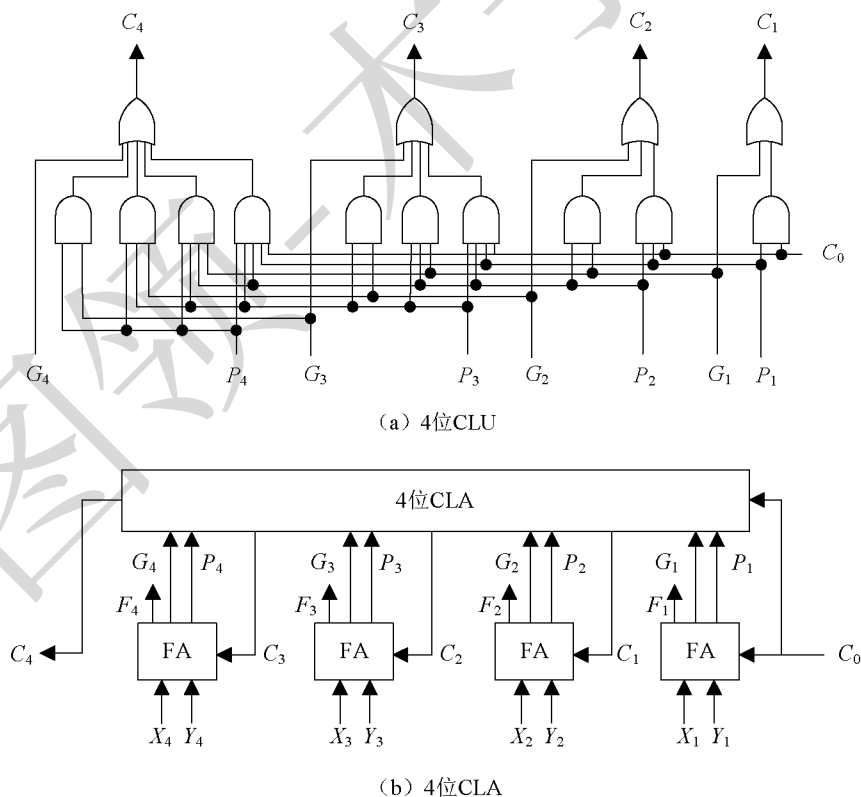


图 3.5 不同层次语言之间的等价转换

对比王道 2.2.2 定点数的移位运算。

（一）未解释算术移位的另一种移位方式

算术移位的规则有两种方式，一种是符号位保持不变，数值位进行移位；二是符号位和数值位都参与移位。

对于第一种：

a) 当真值为正数时，其原码、反码和补码都相同，算术移位后，出现的空位，规定添补 0。即左移时，高位移出，低位补 0，若最高位丢 1，则结果出错；右移时低位移出，高位补 0，若最低位丢 1，则精度缺失。

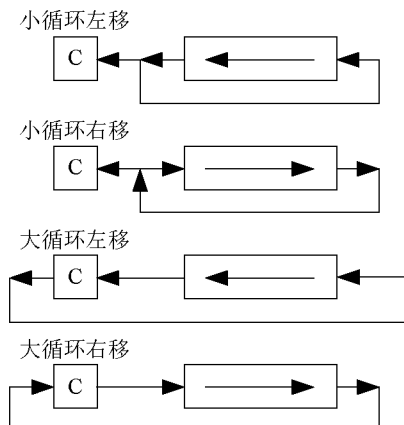
b) 当真值为负数时，真值的原码、反码和补码都不同，（这里仅讨论补码，因为带符号整数在机器内部都用补码表示）。左移时，高位移出，低位补 0，若最高位丢 0，则结果出错。右移时，低位移出，高位补 1。若最低位丢 1，则影响精度。

对于第二种：

若左移时，高位移出，低位补 0，移出前后符号位发生变化，则发生溢出；若右移时，低位移出，高位补符号位。最低位移出 1，则精度缺失。

（二）、未补充无符号数的循环移位的方式和规则。

对于 n 位无符号数来说，还有一种移位操作为循环移位，即其各个位向左或向右移动时，被移出的位会重新出现在另一端，形成循环。循环移位分为小循环左移或右移，也叫不带 CF 标志位的循环移位，以及大循环左移或右移，也叫带标志位的循环移位，如下图所示。



其中：

①不带进位左循环移位的规则为：将所有位向左移动 k 位，最高位移出的位同时补到最低位和进位标志位 CF，原 CF 的值被丢弃，进位标志位 CF 等于最后一次移出的最高位。

②不带进位右循环移位的规则为：将所有位向右移动 k 位，最低位移出的位同时补到最高位和进位标志位 CF，原 CF 的值被丢弃，进位标志位 CF 等于最后一次移出的最低位。

③带进位左循环移位的规则为：将进位标志位 CF 作为数值的第 $n+1$ 位参与循环，所有位向左移动 k 位，最高位移入 CF，原 CF 移入最低位。

④带进位右循环移位的规则为：将进位标志位 CF 作为数值的第 $n+1$ 位参与循环，所有位向右移动 k 位，最低位移入 CF，原 CF 移入最高位。

对比王道 2.2.4 定点数的乘除运算

（一）未介绍原码一位乘法的实现的例子

例 3.2 已知 $[x]_{\text{原}}=0.1101$ ， $[y]_{\text{原}}=0.1011$ ，用原码一位乘法计算 $[x \times y]_{\text{原}}$ 。

解：先采用无符号数乘法计算 1101×1011 的乘积，原码一位乘法过程如下。

C	P	Y	说明
0	0000	1011	$P_0=0$
+ 1101			$Y_4=1, +X$
0	1101		C, P 和 Y 同时右移一位
0	0110	1101	得 P_1
+ 1101			$Y_3=1, +X$
1	0011		C, P 和 Y 同时右移一位
0	1001	1110	得 P_2
+ 0000			$Y_2=0$ ，不做加法（加0）
0	1001	1110	C, P 和 Y 同时右移一位
0	0100	1111	得 P_3
+ 1101			$Y_1=1, +X$
1	0001		C, P 和 Y 同时右移一位
0	1000	1111	得 P_4

符号位为 $0 \oplus 0 = 0$ ，因此， $[x \times y]_{\text{原}}=0.10001111$ 。

例 3.10 已知 $x=0.1101$ ， $y=-0.1011$ ，用原码一位乘法求 $x \times y$ 。

解：

部分积	乘数 $ y $	说明
00.0000	1011	$P_0=0$
+ 00.1101		$y_n=1, + x $
00.1101	1011	
→ 00.0110	1101	右移一位, 得 P_1
+ 00.1101		$y_n=1, + x $
01.0011	1101	
→ 00.1001	1110	右移一位, 得 P_2
+ 00.0000		$y_n=0, +0$
00.1001	1110	
→ 00.0100	1111	右移一位, 得 P_3
+ 00.1101		$y_n=1, + x $
01.0001	1111	
→ 00.1000	1111	右移一位, 得 $P_4= x \times y $

由于 $P_f = x_0 \oplus y_0 = 0 \oplus 1 = 1$, 因此 $[x \times y]_{\text{原}} = 1.10001111$, $x \times y = -0.10001111$ 。

例 6.17 已知 $x = -0.1110$, $y = -0.1101$, 求 $[x \cdot y]_{\text{原}}$ 。

解：因为 $x = -0.1110$

所以 $[x]_{\text{原}} = 1.1110$, $x^* = 0.1110$ (为绝对值), $x_0 = 1$

又因为 $y = -0.1101$

所以 $[y]_{\text{原}} = 1.1101$, $y^* = 0.1101$ (为绝对值), $y_0 = 1$

按原码一位乘运算规则, $[x \cdot y]_{\text{原}}$ 的数值部分计算如表 6.9 所示。

表 6.9 例 6.17 数值部分的计算

部分积	乘数	说明
0.0000	1101	开始部分积 $z_0=0$
+ 0.1110		乘数为 1, 加上 x^*
0.1110		
0.0111	0110	→1 位得 z_1 , 乘数同时 →1 位
+ 0.0000		乘数为 0, 加上 0
0.0111	0	
0.0011	1011	→1 位得 z_2 , 乘数同时 →1 位
+ 0.1110		乘数为 1, 加上 x^*
1.0001	10	
0.1000	1101	→1 位得 z_3 , 乘数同时 →1 位
+ 0.1110		乘数为 1, 加上 x^*
1.0110	110	
0.1011	0110	→1 位得 z_4 , 乘数已全部移出

即 $x^* \cdot y^* = 0.10110110$, 乘积得符号位为 $x_0 \oplus y_0 = 1 \oplus 1 = 0$, 故 $[x \cdot y]_{\text{原}} = 0.10110110$ 。

(二) 未介绍补码一位乘法的运算实现和举例说明。

补码一位乘法的运算规则如下：

- (1) 乘数最低位增加一位辅助位 $Y_{-1}=0$;
- (2) 根据 Y_iY_{i-1} 的值，决定是 “ $+ [x]_{\text{补}}$ ” “ $- [x]_{\text{补}}$ ” 还是 “ $+0$ ”;
- (3) 每次加减后，算术右移一位，得到部分积;
- (4) 重复第 (2) 和第 (3) 步 n 次，结果得 $[x \times y]_{\text{补}}$ 。

例 3.3 已知 $[x]_{\text{补}}=1\ 101$, $[y]_{\text{补}}=0\ 110$, 要求用布斯乘法计算 $[x \times y]_{\text{补}}$ 。

解： $[-x]_{\text{补}}=0\ 011$, 布斯乘法过程如下。

P	Y	Y_{-1}	说 明
0000	0110	0	设 $Y_{-1}=0$, $[P_0]_{\text{补}}=0$
0000	0011	0	$Y_0Y_{-1}=00$, P 、 Y 直接右移一位 得 $[P_1]_{\text{补}}$
+0011			$Y_1Y_0=10$, $+ [-x]_{\text{补}}$
0011			P 、 Y 同时右移一位
0001	1001	1	得 $[P_2]_{\text{补}}$
0000	1100	1	$Y_2Y_1=11$, P 、 Y 直接右移一位 得 $[P_3]_{\text{补}}$
+1101			$Y_3Y_2=01$, $+ [x]_{\text{补}}$
1101			P 、 Y 同时右移一位
1110	1110	0	得 $[P_4]_{\text{补}}$

因此, $[x \times y]_{\text{补}}=1110\ 1110$ 。

例 3.11 设 $[x]_{\text{补}}=1.0111$, $[y]_{\text{补}}=1.0011$, 求 $[x \times y]_{\text{补}}$ 。

解： $[-x]_{\text{补}}=0.1001$

部分积	乘数 y	说明
00.0000	100110	$y_{n+1}=0\ P_0=0$
+ 00.1001		$y_n y_{n+1}=10$, $+ [-x]_{\text{补}}$
00.1001		
→ 00.0100	110011	右移一位, 得 P_1
+ 00.0000		$y_n y_{n+1}=11$, $+0$
00.0100		
→ 00.0010	111001	右移一位, 得 P_2
+ 11.0111		$y_n y_{n+1}=01$, $+ [x]_{\text{补}}$
11.1001		
→ 11.1100	101100	右移一位, 得 P_3
+ 00.0000		$y_n y_{n+1}=00$, $+0$
11.1100		
→ 11.1110	010110	右移一位, 得 P_4
+ 00.1001		$y_n y_{n+1}=10$, $+ [-x]_{\text{补}}$
00.0111	0101	最后一步数据不移位

所以 $[x \times y]_n = 0.0111\ 0101$ 。

(三) 未介绍原码一位除法的运算实现步骤和例子。

参考手工除法过程，得到计算机中两个无符号数除法的运算步骤和算法要点如下。

(1) 操作数预置。在确认被除数和除数都不为 0 后，将被除数（必要时进行 0 扩展）置于余数寄存器 R 和余数/商寄存器 Q 中，除数置于除数寄存器 Y 中。

(2) 做减法试商。根据 $R-Y$ 的符号来判断两数大小。若为正，则上商 1；若为负，则上商 0。

(3) 上商为 0 时恢复余数。把减掉的除数再加回来，恢复原来的中间余数。

(4) 中间余数左移，以便继续试商。手算除法中，每次试商前，除数右移后，与中间余数进行比较。在计算机内部进行除法运算时，除数在除数寄存器中不动，因此，需要将中间余数左移，将左移结果与除数相减以进行比较。左移时中间余数和商一起左移， Q 的最低位空出，以备上商。

上述算法要点 (3) 中，采用了“上商为 0 时恢复余数”的方式，所以，把上述这种方法称为“恢复余数法”。也可以不这样做，而是在下一步运算时把当前多减的除数补回来。这种方法称为“不恢复余数法”，又称“加减交替法”。根据余数恢复方式的不同，有“恢复余数除法”和“不恢复余数除法”两种。

1. 恢复余数除法

假定被除数 X 为 $2n$ 位，除数 Y 和商 Q 都为 n 位， X 、 Y 和 Q 分别表示为： $X = X_{2n-1}X_{2n-2} \cdots X_nX_{n-1} \cdots X_0$ ， $Y = Y_{n-1}Y_{n-2} \cdots Y_0$ ， $Q = Q_{n-1}Q_{n-2} \cdots Q_0$ ，则恢复余数除法的算法步骤如下。

第 1 步， $R_1 = X - Y$ ，若 $R_1 < 0$ ，则上商 $Q_n = 0$ ，同时恢复余数，即 $R_1 = R_1 + Y$ ；若 $R_1 \geq 0$ ，则上商 $Q_n = 1$ 。这里求得的商 Q_n 是商的第 n 位数值。显然，若 $Q_n = 1$ ，则商将会有 $n+1$ 位数。这对于以下不同的情况，意味着不同的结果。

(1) 对于无符号整数除法来说，如果被除数为 $2n$ 位，则商有可能会超出 n 位无符号整数范围，所以，若 $Q_n = 1$ ，则发生溢出。

(2) 对于原码定点小数除法来说，若 $Q_n = 1$ ，则相除结果的数值从小数部分溢出了到整数部分，按道理两个定点小数相除，结果也应是定点小数，故应当作溢出处理。但浮点

数尾数溢出时，可通过右规来消除，最终只要阶码不溢出，结果仍然正确。所以，这种情况下，保留最高位的商 $Q_n=1$ ，继续执行下去。

第 2 步，若已求得第 i 次的中间余数为 R_i ，则第 $i+1$ 次的中间余数为 $R_{i+1}=2R_i-Y$ 。若 $R_i < 0$ ，则上商 $Q_{n-i}=0$ ，同时恢复余数，即 $R_{i+1}=R_{i+1}+Y$ ；若 $R_{i+1} \geq 0$ ，则上商 $Q_{n-i}=1$ 。

第 3 步，循环执行第 2 步 n 次，直到求出所有 n 位商 $Q_{n-1} \sim Q_0$ 为止。

最终商在 Q 寄存器中、余数在 R 寄存器中。

例 3.4 已知 $[x]_{\text{原}}=0.1011$ ， $[y]_{\text{原}}=1.1101$ ，用恢复余数法计算 $[x/y]_{\text{原}}$ 。

解：分符号位和数值位两部分进行。

商的符号位： $0 \oplus 1 = 1$ 。

商的数值位采用恢复余数法。减法操作用补码加法实现，是否够减通过中间余数的符号来判断，所以中间余数要加一位符号位。因此，需先计算出 $[x]_{\text{补}}=0.1011$ ， $[y]_{\text{补}}=0.1101$ ， $[-y]_{\text{补}}=1.0011$ 。

因为是原码定点小数，所以在低位扩展 0。虽然实际参加运算的是 $[x]_{\text{补}}$ 和 $[y]_{\text{补}}$ ，但为简单起见，以下说明中分别标识为 X 和 Y 。

运算过程如下：

余数寄存器R	余数/商寄存器Q	说明
0 1 0 1 1	0 0 0 0 □	开始 $R_0=X$
+ 1 0 0 1 1		$R_1=X-Y$
1 1 1 1 0	0 0 0 0 0	$R_1 < 0$ ，则 $Q_4=0$
+ 0 1 1 0 1		恢复余数： $R_1=R_1+Y$
0 1 0 1 1		得 R_1
1 0 1 1 0	0 0 0 0 □	$2R_1$ （ R 和 Q 同时左移，空出一位商）
+ 1 0 0 1 1		$R_2=2R_1-Y$
0 1 0 0 1	0 0 0 0 1	$R_2 > 0$ ，则 $Q_3=1$
1 0 0 1 0	0 0 0 1 □	$2R_2$ （ R 和 Q 同时左移，空出一位商）
+ 1 0 0 1 1		$R_3=2R_2-Y$
0 0 1 0 1	0 0 0 1 1	$R_3 > 0$ ，则 $Q_2=1$
0 1 0 1 0	0 0 1 1 □	$2R_3$ （ R 和 Q 同时左移，空出一位商）
+ 1 0 0 1 1		$R_4=2R_3-Y$
1 1 1 0 1	0 0 1 1 0	$R_4 < 0$ ，则 $Q_1=0$
+ 0 1 1 0 1		恢复余数： $R_4=R_4+Y$
0 1 0 1 0	0 0 1 1 0	得 R_4
1 0 1 0 0	0 1 1 0 □	$2R_4$ （ R 和 Q 同时左移，空出一位商）
+ 1 0 0 1 1		$R_5=2R_4-Y$
0 0 1 1 1	0 1 1 0 1	$R_5 > 0$ ，则 $Q_0=1$

商得最高位为 0，说明没有溢出，商得数值部分为 1101。所以， $[x/y]_{\text{原}}=1.1101$ （最高位为符号位），余数为 0.0111×2^{-4} 。

2. 不恢复余数除法

在恢复余数除法运算中，当中间余数与除数相减结果为负时，要多做一次+Y 操作，因而降低了算法执行速度，又使控制线路变得复杂。在计算机中很少采用恢复余数除法，而普遍采用不恢复余数除法。其实现原理如下。

在恢复余数除法中，第 i 次余数为 $R_i=2R_{i-1}-Y$ 。根据下次中间余数的计算方法，有以下两种不同情况：

(1) 若 $R_i \geq 0$ ，则上商 1，不需恢复余数，直接左移一位后试商，得下次余数 R_{i+1} ，即 $R_{i+1}=2R_i-Y$ ；

(2) 若 $R_i < 0$ ，则上商 0，且需恢复余数后左移一位再试商，得下次余数 R_{i+1} ，即 $R_{i+1}=2(R_i+Y)-Y=2R_i+Y$ 。

当第 i 次中间余数为负时，可以跳过恢复余数这一步，直接求第 $i+1$ 次中间余数。这种算法称为不恢复余数法。从上述推导可以发现，不恢复余数法的算法要点就是 6 个字：“正、1、减，负、0、加”。其含义就是：若中间余数为正数，则上商为 1，下次做减法；若中间余数为负数，则上商为 0，下次做加法。这样运算中每次循环内的步骤都是规整的，差别仅在做加法还是减法，所以，这种方法也称为“加减交替法”。采用这种方法有一点要注意，即如果在最后一步上商为 0，则必须恢复余数，把试商时减掉的除数加回去。

例 3.5 已知 $[x]_{\text{原}}=0.1011$ ， $[y]_{\text{原}}=1.1101$ ，用不恢复余数法计算 $[x/y]_{\text{原}}$ 。

解：分符号位和数值位两部分进行。

商的符号位： $0 \oplus 1 = 1$ 。

商的数值位采用不恢复余数法。减法操作用补码加法实现，是否够减通过中间余数的符号来判断，所以中间余数要加一位符号位。先计算出 $X=[x]_{\text{补}}=0.1011$ ， $Y=[y]_{\text{补}}=0.1101$ ， $-Y=[-y]_{\text{补}}=1.0011$ 。

运算过程如下：

余数寄存器R	余数/商寄存器Q	说明
0 1 0 1 1	0 0 0 0 □	开始 $R_0=X$
+1 0 0 1 1		$R_1=X-Y$
1 1 1 1 0	0 0 0 0 0	$R_1<0$, 则 $Q_4=0$, 没有溢出
1 1 1 0 0	0 0 0 0 □	$2R_1$ (R 和 Q 同时左移, 空出一位商)
+0 1 1 0 1		$R_2=2R_1+Y$
0 1 0 0 1	0 0 0 0 1	$R_2>0$, 则 $Q_3=1$
1 0 0 1 0	0 0 0 1 □	$2R_2$ (R 和 Q 同时左移, 空出一位商)
+1 0 0 1 1		$R_3=2R_2-Y$
0 0 1 0 1	0 0 0 1 1	$R_3>0$, 则 $Q_2=1$
0 1 0 1 0	0 0 1 1 □	$2R_3$ (R 和 Q 同时左移, 空出一位商)
+1 0 0 1 1		$R_4=2R_3-Y$
1 1 1 0 1	0 0 1 1 0	$R_4<0$, 则 $Q_1=0$
1 1 0 1 0	0 1 1 0 □	$2R_4$ (R 和 Q 同时左移, 空出一位商)
+0 1 1 0 1		$R_5=2R_4+Y$
0 0 1 1 1	0 1 1 0 1	$R_5>0$, 则 $Q_0=1$

商的最高位为 0，说明没有溢出，商的数值部分为 1101。所以， $[x/y]_{\text{原}}=1.1101$ （最高位为符号位），余数为 0.0111×2^{-4} 。

从上述几个除法例子以及有关恢复余数法和不恢复余数法的算法流程可以看出，要得到 n 位无符号数的商，需要循环 $n+1$ 次，其中第一次得到的不是真正的商，而是用来判断溢出的。为了节省运算时间，第一次可以不试商而直接左移，这样只要 n 次循环。因为对于两个 n 位定点整数除法来说，其商一定不会超过 n 位，所以不会发生溢出，因而 n 位定点整数除法第一次无须试商来判断溢出。

【1800 题举例】

49.在原码不恢复余数法（加减交替法）中，若某次运算得到的余数为负数，则下一步应该（ ）。

- A. 商上 0，将余数左移一位后加除数
- B. 商上 1，将余数左移一位后减除数
- C. 商上 0，将余数左移一位后减除数
- D. 商上 1，将余数左移一位后加除数

（四）未介绍补码一位除法的运算实现步骤和例子

补码作为带符号整数的表示形式，需要计算机能实现定点补码整数的除法运算。与补码加减运算、补码乘法运算一样，补码除法也可以将符号位和数值位合在一起进行运算，而且商符直接在除法运算中产生。对于两个 n 位补码除法，被除数需要进行符号扩展。若被除数为 $2n$ 位，除数为 n 位，则被除数无须扩展。

同样，首先要对被除数和除数的取值、大小等进行相应的判断，以确定除数是否为 0，

商是否为 0、是否溢出等。

因为补码除法中被除数、中间余数和除数都是有符号的，所以，不像无符号除法和原码除法那样可以直接用做减法来判断是否够减，而应该根据被除数（中间余数）与除数之间符号的异同或差值的正负来确定下次做减法还是加法，再根据加或减运算的结果来判断是否够减。表 3.2 给出了判断是否够减的规则。

表 3.2 补码除法根据符号判断是否够减的规则

中间余数 R	除数 Y	新中间余数: $R-Y$		新中间余数: $R+Y$	
		0	1	0	1
0	0	够减	不够减		
0	1			够减	不够减
1	0			不够减	够减
1	1	不够减	够减		

从表 3.2 可看出，当被除数（中间余数）与除数同号时做减法；异号时，做加法。若加减运算后得到的新余数与原余数符号一致（余数符号未变）则够减；否则不够减。

根据是否立即恢复余数，补码除法也分为恢复余数法和不恢复余数法两种。

1.补码恢复余数除法

根据补码除法判断是否够减的判断规则，可以得到如下补码恢复余数除法的算法要点。

（1）操作数的预置。除数装入除数寄存器 Y ，被除数符号扩展后装入余数寄存器 R 和余数/商寄存器 Q 。

（2） R 和 Q 同步串行左移一位。

（3）若 R 与 Y 同号，则做减法，即 $R=R-Y$ ；否则，做加法，即 $R=R+Y$ ，并按以下规则确定第 i 次循环得到的商 Q_{n-i} 。

①若 R 和 Q 中的余数=0 或 R 操作前后符号未变，则表示够减， Q_{n-i} 置 1，转第（4）步。

②若 R 操作前后符号已变，表示不够减，则 Q_{n-i} 置 0，恢复 R 值后转第（4）步。

（4）重复第（2）和第（3）步，直到取得 n 位商为止。

（5）若被除数与除数同号，则 Q 中是真正的商；否则，将 Q 中的数值求补后作为商。

（6）余数在 R 中。

从上述算法要点可以看出，在进行置商时，采用了“够减则上商为1，不够减则上商为0”的上商方式，因此，若商为负值，则需要通过“各位取反，末位加1”来得到真正的商。

例 3.6 用 4 位补码恢复余数法计算-7/3 的值。

解：-7 的 8 位补码表示为 $X=[x]_{\text{补}}=1111\ 1001$ ，3 的 4 位补码表示为 $Y=[y]_{\text{补}}=0011$ ， $-Y=[-y]_{\text{补}}=1101$ 。

余数寄存器R	余数/商寄存器Q	说明
1 1 1 1	1 0 0 1	开始 $R_0=X$
1 1 1 1 + 0 0 1 1 ----- 0 0 1 0 + 1 1 0 1 ----- 1 1 1 1	0 0 1 □ 0 0 1 □ 0 0 1 0	$2R_0$ (R和Q同时左移，空出一位商) R与Y异号，加 R符号已变 恢复(减) Q_3 置0，得 R_1
1 1 1 0 + 0 0 1 1 ----- 0 0 0 1 + 1 1 0 1 ----- 1 1 1 0	0 1 0 □ 0 1 0 □ 0 1 0 0	$2R_1$ (R和Q同时左移，空出一位商) R与Y异号，加 R符号已变 恢复(减) Q_2 置0，得 R_2
1 1 0 0 + 0 0 1 1 ----- 1 1 1 1	1 0 0 □ 1 0 0 1	$2R_2$ (R和Q同时左移，空出一位商) R与Y异号，加 R符号未变， Q_1 置1，得 R_3
1 1 1 1 + 0 0 1 1 ----- 0 0 1 0 + 1 1 0 1 ----- 1 1 1 1	0 0 1 □ 0 0 1 □ 0 0 1 0	$2R_3$ (R和Q同时左移，空出一位商) R与Y异号，加 R符号已变 恢复(减) Q_0 置0，得 R_4

因为被除数与除数异号，故商取补为 1110，余数为 1111，即-7/3=-2，余数为-1。

2.补码不恢复余数除法

根据补码除法判断是否够减的判断规则，可以得到如下补码不恢复余数除法的算法要点。

(1) 操作数的预置。除数装入除数寄存器 Y，被除数符号扩展后装入余数寄存器 R 和余数/商寄存器 Q。

(2) 根据以下规则求第一位商 Q_n 。若 X 与 Y 同号，则做减法，即 $R_1=X-Y$ ；否则，做加法，即 $R_1=X+Y$ ，并按以下规则确定商值 Q_n 。

- ①若新的中间余数 R_1 与 Y 同号，则 Q_n 置 1，转第 (3) 步。
- ②若新的中间余数 R_1 与 Y 异号，则 Q_n 置 0，转第 (3) 步。

Q_n 用来判断是否溢出，而不是真正的商。以下情况下会发生溢出： X 与 Y 同号且上商 $Q_n=1$ ，或者， X 与 Y 异号且上商 $Q_n=0$ 。

(3) 对于 $i=1$ 到 n ，按以下规则求出相应商。

①若 R_i 与 Y 同号，则 Q_{n-i} 置 1， $R_{i+1}=2R_i-Y$ ， $i=i+1$ 。

②若 R_i 与 Y 异号，则 Q_{n-i} 置 0， $R_{i+1}=2R_i+Y$ ， $i=i+1$ 。

(4) 商的修正。最后一次 Q 寄存器左移一位，将最高位 Q_n 移出，并在最低位置上商 Q_0 。若被除数与除数同号，则 Q 中就是真正的商；否则，将 Q 中的商的末位加 1。

(5) 余数的修正。若余数符号同被除数符号，则不需修正，余数在 R 中；否则，按下列规则进行修正：当被除数和除数符号相同时，最后余数加除数；否则，最后余数减除数。

与无符号数的不恢复余数法一样，补码不恢复余数法也有一个六字口诀“同、1、减，异、0、加”，运算过程也是呈加/减交替的方式，因此也称为“加减交替法”。

例 3.7 已知 $x=-9$ ， $y=2$ ，要求用补码除法计算 $[x/y]_{\text{补}}$ 。

解： $X=[x]_{\text{补}}=1\ 0111$ ， $Y=[y]_{\text{补}}=0\ 0010$ ，计算过程如下：

先对被除数进行符号扩展，即 $X=11111\ 10111$ ， $-Y=[-y]_{\text{补}}=1\ 1110$ 。

余数寄存器R	余数/商寄存器Q	说明
1 1 1 1 1	1 0 1 1 1	开始 $R_0=X$
+ 0 0 0 1 0		$R_1=X+Y$
0 0 0 0 1	1 0 1 1 1	R_1 与 Y 同号，故 $Q_5=1$
0 0 0 1 1	0 1 1 1 1	$2R_1$ (R 和 Q 同时左移，空出一位商1)
+ 1 1 1 1 0		$R_2=2R_1-Y$
0 0 0 0 1	0 1 1 1 1	R_2 与 Y 同号，故 $Q_4=1$
0 0 0 1 0	1 1 1 1 1	$2R_2$ (R 和 Q 同时左移，空出一位商1)
+ 1 1 1 1 0		$R_3=2R_2-Y$
0 0 0 0 0	1 1 1 1 1	R_3 与 Y 同号，故 $Q_3=1$
0 0 0 0 1	1 1 1 1 1	$2R_3$ (R 和 Q 同时左移，空出一位商1)
+ 1 1 1 1 0		$R_4=2R_3-Y$
1 1 1 1 1	1 1 1 1 1	R_4 与 Y 同号，故 $Q_2=0$
1 1 1 1 1	1 1 1 1 0	$2R_4$ (R 和 Q 同时左移，空出一位商0)
+ 0 0 0 1 0		$R_5=2R_4+Y$
0 0 0 0 1	1 1 1 1 0	R_5 与 Y 同号，故 $Q_1=1$
0 0 0 1 1	1 1 1 0 1	$2R_5$ (R 和 Q 同时左移，空出一位商1)
+ 1 1 1 1 0		$R_6=2R_5-Y$
0 0 0 0 1	1 1 0 1 1	Q 左移，空出一位上商。
+ 1 1 1 1 0		R_6 与 Y 同好，故 $Q_0=1$
1 1 1 1 1	1 1 1 0 0	商为负数，末位加1；减除数以修正余数

所以， $[x/y]_{\text{补}}=11100$ ，余数为 11111。

即 $x/y = -0100B = -4$ ，余数为 $-0001B = -1$ 。

将各数代入公式“除数 $\times$ 商+余数=被除数”进行验证，得 $2 \times (-4) + (-1) = -9$ 。

(四)、未介绍阵列乘法器的具体结构和实现原理。

【图领强化讲义】

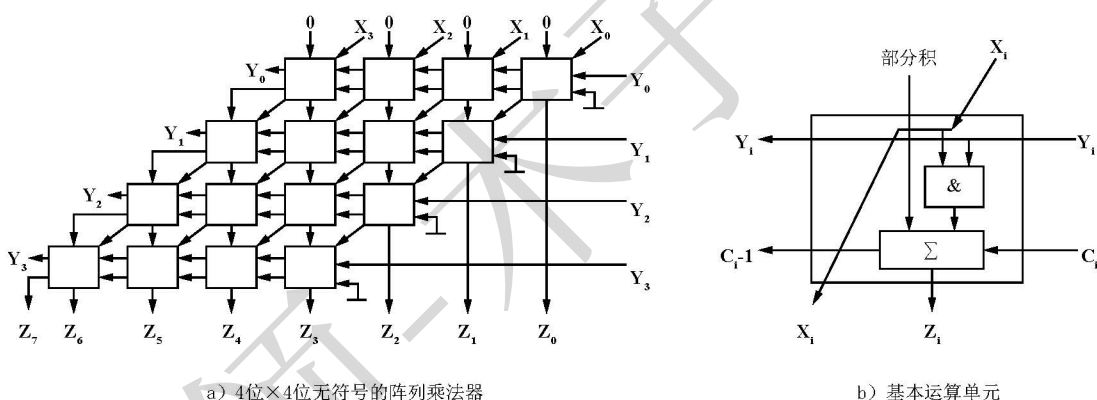
【考察方式】选择题、大题

【深入剖析】

该部分相关的运算原理在 1.2 部分介绍过，这里不再叙述。而是补充一下阵列乘法器的相关概念和原理。

阵列乘法器是一种并行化乘法电路，通过硬件阵列实现多位乘法的快速计算，相比串行乘法器大幅提升运算速度，通常可在一个时钟周期内完成乘运算。

如下图所示为 4×4 无符号阵列乘法器结构。



a) 4位 $\times$ 4位无符号的阵列乘法器

b) 基本运算单元

其中，每个运算单元包含 1 个与门和 1 位全加器，输入为被乘数位 X_i 、乘数位 Y_i 、进位 C_i ，输出为 X_i 、 Y_i 和 C_{i+1} 。

每一行将 4 位乘数划分成一行(组)，具体地由乘数 Y 的每一位数来控制，用于乘运算的一个步骤。而各行错开形成的每一斜列则由被乘数 X 的每一位数控制，这样完成 $X \times Y$ 乘法运算($X=X_3X_2X_1X_0$, $Y=Y_3Y_2Y_1Y_0$)，只用四步即可完成。

若在绝对值阵列乘法器的基础上，可以适当地增加三个求补码器等逻辑电路就可实现带符号数的阵列乘法器。基本思想是先将被乘数与乘数分别经过求补码器后，进行绝对值乘法计算，其结果还要经过求补码器。最后根据被乘数与乘数的符号，决定最后乘积的符号。

个人补充：

串行阵列乘法器本质是采用一维线性阵列结构，通过时间换空间，逐位或逐组处理乘法运算。其工作流程如下：

数据串行输入：被乘数和乘数以位串行方式输入到线性处理单元（PE）阵列。

逐位生成部分积：每个时钟周期处理一位或几位数据，生成对应部分积。

流水累加：部分积在 PE 之间传递并累加，经过 n 个或更多时钟周期后输出完整乘积。

【1800 题举例】

45. 关于阵列乘法器，下列说法正确的是（ ）。

- A. 并行阵列乘法器在计算两个 n 位二进制数相乘时，不需要考虑进位的传递
- B. 串行阵列乘法器是逐位依次完成乘法运算，其速度比并行阵列乘法器快
- C. 对于 n 位的并行阵列乘法器，其输出结果的位数一定是 n 位
- D. 带符号数的阵列乘法器通常采用补码进行运算以简化硬件设计

46. 【2024 年统考真题】下列关于整数乘法运算的叙述中，错误的是（ ）。

- A. 用阵列乘法器实现的乘运算可以在一个时钟周期内完成
- B. 用 ALU 和移位器实现的乘运算无法在一个时钟周期内完成
- C. 变量与常数的乘运算可编译优化为若干移位及加/减运算指令
- D. 两个变量的乘运算无法编译转换为移位及加法等指令的循环实现

第三章

3.2 主存储器

对比王道 3.2.1 半导体随机存取存储器。

（一）未给出 DRAM 的刷新方式

常见的刷新方式有集中式、分散式和异步式 3 种。

（1）集中刷新方式

在允许的最大刷新间隔（如 2 ms）内，按照存储芯片容量的大小集中安排若干个刷新周期，刷新时停止读写操作。

刷新时间=存储矩阵行数×刷新周期

这里刷新周期是指刷新一行所需要的时间，由于刷新过程就是“假读”的过程，所以刷新周期就等于存取周期。

例如，对具有 1024 个记忆单元（排列成 32×32 的存储矩阵）的存储芯片进行刷新，刷新是按行进行的，且每刷新一行占用一个存取周期，所以共需 32 个周期以完成全部记忆单元的刷新。假设存取周期为 500 ns，则在 2 ms 内共可以安排 4000 个存取周期，从 0~3967 个周期内进行读写操作或保持，而从 3968~3999 最后 32 个周期集中安排刷新操作如图 3-2 所示。

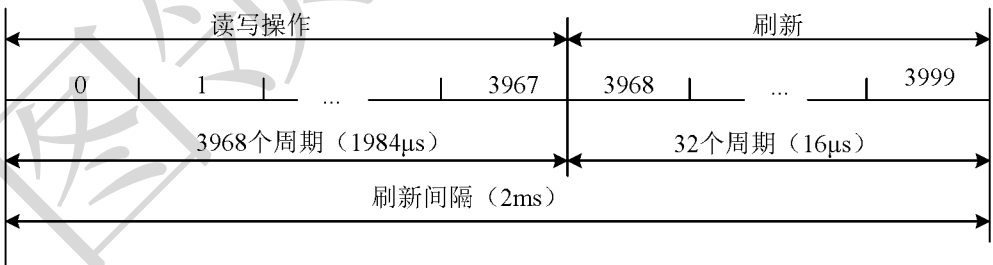


图 3-2 集中刷新方式

集中刷新方式的优点是读写操作时不受刷新工作的影响，因此系统的存取速度比较高。主要缺点是在集中刷新期间必须停止读写，这一段时间称为“死区”，而且存储容量越大，死区就越长。

(2) 分散刷新方式

分散刷新是指把刷新操作分散到每个存取周期内进行，此时系统的存取周期被分为两部分，前一部分时间进行读写操作或保持，后一部分时间进行刷新操作。在一个系统存取周期内刷新存储矩阵中的一行。这种刷新方式增加了系统的存取周期，如存储芯片的存取周期为 0.5 us，则系统的存取周期应为 1 us，即前一个 0.5 us 读写，后一个 0.5 us 刷新。仍以前述的 32×32 矩阵为例，整个存储芯片刷新一遍需要 32 us，如图 3-3 所示。

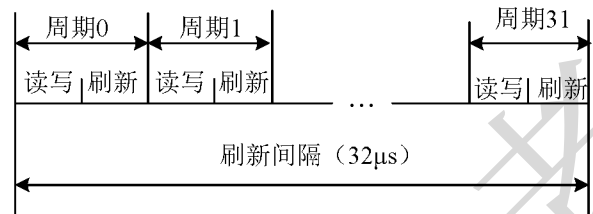


图 3-3 分散刷新方式

这种刷新方式没有死区，但是它也有很明显的缺点，一是加长了系统的存取周期，降低了整机的速度；二是刷新过于频繁（本例中每 32 us 就重复刷新一遍），尤其是当存储容量比较小的情况下，没有充分利用所允许的最大刷新闻隔。

(3) 异步刷新方式

这种刷新方式可以看成前述两种方式的结合，它充分利用了最大刷新闻隔时间，把刷新操作平均分配到整个最大刷新闻隔时间内进行，故有：

相邻两行的刷新闻隔=最大刷新闻隔时间÷行数

对于 32×32 矩阵，在 2 ms 内需要将 32 行刷新一遍，所以相邻两行的刷新时间间隔= $2\text{ ms} \div 32 = 62.5\text{ us}$ ，即每隔 62.5 us 安排一个刷新周期。在刷新时封锁读写，如图 3-4 所示。

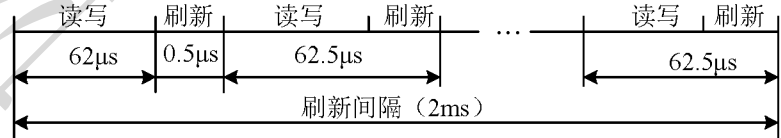


图 3-4 异步刷新方式

异步刷新方式虽然也有死区，但比集中刷新方式的死区小得多，仅为 0.5 us。这样可以避免使 CPU 连续等待过长的时间，而且减少了刷新次数，是比较实用的一种刷新方式。

【1800 题举例】

考点角度 1: DRAM 的刷新

4. 已知某 DRAM 芯片, 其存储矩阵为 128×128 结构, 若采用异步刷新方式, 且刷新间隔为 2 ms , 则该 DRAM 芯片的刷新周期是 ()。

- A. $15.625\text{ }\mu\text{s}$ B. $31.25\text{ }\mu\text{s}$ C. $62.5\text{ }\mu\text{s}$ D. $125\text{ }\mu\text{s}$

5. 【2015 年统考真题】下列存储器中, 在工作期间需要周期性刷新的是 ()。

- A. SRAM B. SDRAM C. ROM D. Flash

6. 以下对于存储器刷新操作的描述中, 正确的是 ()。

- A. DRAM 和 SRAM 都需要刷新
B. 刷新是按行进行的
C. 刷新是按一个芯片接着一个芯片的顺序进行的
D. 所有的刷新方式都存在“死区”

7. 对于某存储芯片, 假定其动态刷新间隔为 2 ms , 读写周期和刷新周期均为 $0.5\text{ }\mu\text{s}$, 该芯片中包含 128 行, 每个刷新周期可以完成 1 行存储单元的刷新, 如果该芯片采用异步刷新方式工作, 那么其读写周期和刷新周期可以安排为 ()。

- A. 3999 次读写周期后, 安排 1 次刷新操作
B. 2000 次读写周期后, 安排 1 次刷新操作
C. 128 次读写周期后, 安排 1 次刷新操作
D. 30 次读写周期后, 安排 1 次刷新操作

8. 对于某 DDR 存储芯片, 需要动态刷新保持数据, 假定其刷新间隔为 2 ms , 读写周期和刷新周期均为 $0.5\text{ }\mu\text{s}$, 该芯片中包含 128 行, 每个刷新周期可完成 1 行存储单元的刷新, 如果该芯片采用集中刷新的方式工作, 那么读写周期和刷新周期的合理安排为 ()。

- A. 4000 次读写周期后, 安排 128 次刷新操作
B. 3872 次读写周期后, 安排 128 次刷新操作
C. 128 次读写周期后, 安排 1 次刷新操作
D. 2000 次读写周期后, 安排 1 次刷新操作

（二）未解释 DRAM 芯片的双译码方式和具体结构

目前，动态存储芯片大多采用双译码结构。地址译码器分为 X 和 Y 方向两个译码器。

图 7.5 采用的就是二维双译码结构，其存储阵列组织如图 7.6 所示。

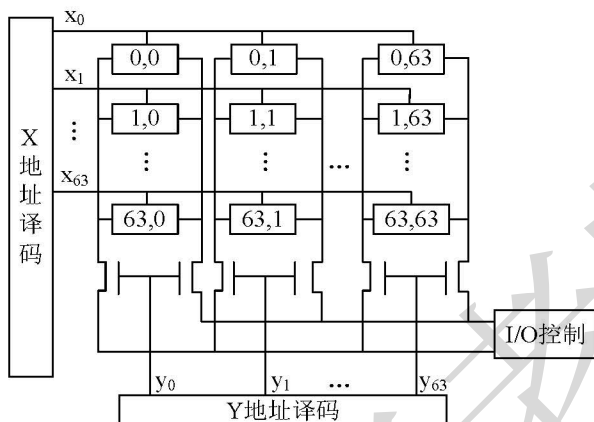


图 7.6 二维双译码结构（位片式芯片）

图 7.6 中的存储阵列有 4096 个单元，需要 12 根地址线： $A_{11} \sim A_0$ ，其中 $A_{11} \sim A_6$ 送至 X 译码器，有 64 条译码输出线，各选择一行单元； $A_5 \sim A_0$ 送至 Y 译码器，也有 64 条译码输出线，分别控制一列单元的位线控制门。假如输入的 12 位地址为 $A_{11}A_{10} \cdots A_0 = 000001000000$ 时，则 X 译码器的第 2 根译码输出线（ x_1 ）为高电平，于是与它相连的 64 个存储单元的字选择 W 线为高电平。Y 译码器的第 1 根译码输出线（ y_0 ）为高电平，打开第一列的位线控制门。在 X、Y 译码的联合作用下，存储矩阵中 (1,0) 单元被选中。

在选中的行和列交叉点上的单元只有一位，因此，采用二维双译码结构的存储器芯片被称为位片式芯片。有些芯片的存储阵列采用三维结构，用多个位平面构成存储阵列，不同位平面在同一行和列交叉点上的多位构成一个存储字，被同时读出或写入。

【1800 题举例】

11. 【2022 年统考真题】某内存条包含 8 个 $8192 \times 8192 \times 8$ 位的 DRAM 芯片，按字节编址，支持突发（burst）传送方式，对应存储器总线宽度为 64 位，每个 DRAM 芯片内有一个行缓冲区（rowbuffer）。下列关于该内存条的叙述中，不正确的是（ ）。

- A. 内存条的容量为 512 MB
- B. 采用多模块交叉编址方式
- C. 芯片的地址引脚为 26 位
- D. 芯片内行缓冲有 8192×8 位

12. 假设有 8 个 $16\text{ M} \times 8$ 位的 DRAM 芯片扩展构成一个 $16\text{ M} \times 64$ 位内存条，且支持突发传送方式，则下列说法正确的是（ ）。

I. 该内存条中，所有芯片行缓冲的总大小为 32 KB

II. 采用多模块交叉编址方式

III. 每个芯片的地址引脚和数据引脚总数为 20 位

IV. 若使用该内存条的计算机字长为 64 位，存储器总线的宽度为 8 位，采用 64 位定长指令字，则主存数据寄存器（MDR）的宽度为 64 位

A. I、III

B. I、II、III

C. II、III、IV

D. II、IV

（三）未介绍 Flash 存储器的原理和基本操作

Flash 存储器也称为闪存，是高密度非易失性读写存储器，它兼有 RAM 和 ROM 的优点，而且功耗低、集成度高，不需后备电源。这种器件沿用了 EPROM 的简单结构和浮栅/热电子注入的编程写入方式，又兼备 EEPROM 的可擦除特点，且可在计算机内进行擦除和编程写入，因此又称为快擦型 EEPROM。目前被广泛使用的 U 盘和存储卡等都属于 Flash 存储器。

1. Flash 存储元

Flash 存储元是在 EPROM 存储元基础上发展起来的。如图 7.18 所示是一个 Flash 存储元，每个存储元由单个 MOS 管组成，包括漏极 D、源极 S、控制栅和浮空栅。当控制栅加上足够的正电压时，浮空栅将储存大量电子，即带有许多负电荷，可将存储元的这种状态定义为 0；当控制栅不加正电压，则浮空栅少带或不带负电荷，将这种状态定义为 1。

2. Flash 存储器的基本操作

闪存有 3 种基本操作：编程（充电）、擦除（放电）、读取。

编程操作：最初所有存储元都是 1 状态，通过编程，在需要改写为 0 的存储元的控制栅加上一个正电压 V_P ，如图 7.19(a)所示。一旦某存储元被编程，则存储的数据可保持 100 年而无须外电源。

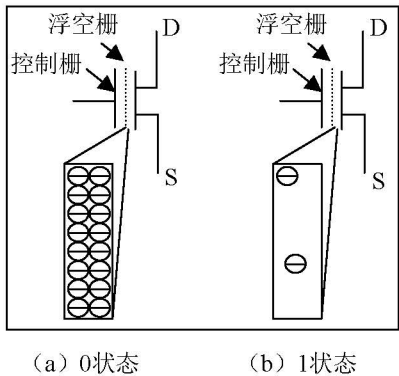


图 7.18 Flash 存储元

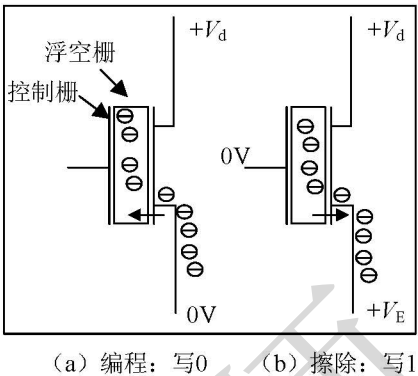


图 7.19 Flash 存储元的写入

擦除操作：采用电擦除。即在所有存储元的源极 S 加正电压 V_E ，使浮空栅中的电子被吸收掉，从而使所有存储元都变成 1 状态，如图 7.19 (b) 所示。因此，写的过程实际上是先全部擦除，使全都变成 1 状态后再在需要的地方改写为 0，即先全部放电，再在写 0 的地方充电。

读取操作：在控制栅加上正电压 V_R ，若原存为 0，则如图 7.20 (a) 所示，读出电路检测不到电流；若原存为 1，则如图 7.20 (b) 所示，浮空栅不带负电荷，控制栅上的正电压足以开启晶体管，电源 V_d 提供从漏极 D 到源极 S 的电流，读出电路检测到电流。

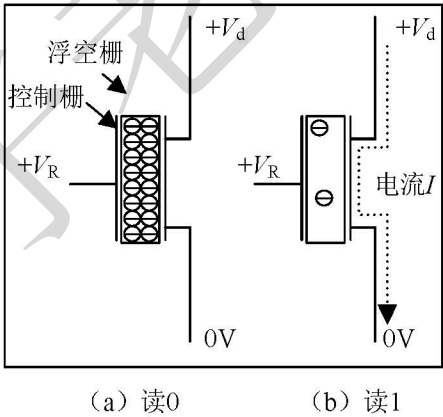


图 7.20 Flash 存储元的读出

从上述基本原理可以看出，Flash 存储器的读操作速度和写操作速度相差很大，其读取速度与半导体 RAM 芯片相当，而写数据(擦除-编程)的速度则比 RAM 芯片慢很多。

3.3 主存储器与 CPU 的连接

对比王道 3.3.2 主容量的拓展

(一) 未介绍全译码法和部分译码法的区别

【图领强化讲义】

1.全译码法

全译码法将除片内寻址外的全部高位地址线都作为地址译码器的输入，译码器的输出

作为各芯片的片选信号，将它们分别接到存储芯片的片选端，以实现存储芯片的选择。

全译码法的优点是每片（或组）芯片的地址范围是唯一确定的，而且是连续的，也便于扩展，不会产生地址重叠的存储区，但全译码法对译码电路要求较高。

如 CPU 的地址总线有 20 位，现用 4 片 2 K×8 的存储芯片组成一个 8 K×8 的存储器。全译码法要求除去片内寻址用到的 11 位地址线外，高 9 位地址 A₁₉~A₁₁ 都要参与译码。各芯片的地址范围如下表所示。

芯片	A ₁₉ ~A ₁₃	A ₁₂ A ₁₁	A ₁₀ ~A ₀	地址范围(空间)
0#	0...0	00	00...0 ... 11...1	00000~007FFH
1#	0...0	01	00...0 ... 11...1	00800~00FFFH
2#	0...0	10	00...0 ... 11...1	01000~017FFH
3#	0...0	11	00...0 ... 11...1	01800~01FFFH

2.部分译码法

所谓部分译码即用除片内寻址外的高位地址的一部分来译码产生片选信号。如，假设地址线 20 位，用 4 片 2 K×8 的存储芯片组成 8 K×8 存储器，需要 4 个片选信号，因此只需要用两位地址线来译码产生。即 A₁₂A₁₁=00，表示选择第一片，A₁₂A₁₁=01 表示选中第二片，依次类推。而只需要令高位 A₁₉~A₁₃ 地址全为 0（也可以全为 1），这样确定的存储器的地址为基本地址，部分译码相比于全译码法比较简单，但存在地址重叠区。

【1800 题举例】

34.某计算机地址空间大小为 2³²B，按字节编址，采用部分译码方式产生片选信号。若片选信号为 1001 时，选定某 32 K×8 的存储芯片工作，则该芯片在存储器中的表示的最大

首地址和最小末地址分别为（ ）。

- A. 0004 8000H, 0004 FFFFH B. FFFC 8000H, 0004 FFFFH
C. FFFC 8000H, FFFC FFFFH D. 0004 8000H, FFFC FFFFH

3.4 外部存储器

对比王道 3.4.1 磁盘存储器

（一）未介绍磁盘阵列的具体原理和特点

【图领讲义】

磁盘阵列是由很多块独立的磁盘，组合成一个容量巨大的磁盘组，利用个别磁盘提供数据所产生加成效果提升整个磁盘系统效能。以下是几种常见的磁盘阵列。

（1）RAID 0

将连续数据块依次存储在多个不同磁盘上，不同磁盘上相同位置的数据块构成一个条带，其中的每个数据块都称为条带单元，即条带化技术。数据可以同时从多个磁盘中读取和写入，从而提高了数据的读写速度。

RAID0 具有如下技术特点：

- ①无数据冗余、无数据校验功能，因此它不具备容错能力，可靠性低。
- ②采用交叉编址的方式进行数据分布，多磁盘可并行工作，可有效提升存储性能。
- ③磁盘利用率高，所有磁盘存储空间都用于保存有效数据。

RAID0 主要应用于对访问性能要求高，但对数据的可靠性要求不高的场合。

（2）RAID 1

通过将数据同时复制到两个或多个磁盘上，实现数据的冗余存储。在写入数据时，数据会同时写入到所有的镜像磁盘中，即镜像磁盘阵列。在读取数据时，可以从任意一个磁盘中读取，读性能最大可提升一倍。

RAID1 具有如下技术特点。

- ①当一个磁盘被损坏时，数据仍可从另一磁盘获取，因此具有很高的可靠性。
- ②存储系统中磁盘的利用率只有 50%。由于 RAID1 的读性能优于写性能，因此 RAID1 主要应用于对数据的可用性要求高，且读操作所占比例较高的场合。

【图领拓展】

将 RAID1 和 RAID0 两个级别进行组合可以构成 RAID10。两个磁盘首先构建成 RAID1，再采用分条技术构建 RAID0 得到 RAID10。该阵列级别的性能和可靠性均优于 RAID1 和 RAID0，也可以采用类似方法得到 RAID01 级别。

(3) RAID2

采用纠错的海明码的磁盘阵列。将数据按比特级别进行条带化，即将数据按照比特切分，并分散存储到不同的磁盘上。

数据在物理上分散在不同磁盘，但在逻辑上保持连续，在数据读写时可以像访问单个连续存储设备一样进行操作，提高了数据的读取和写入效率。

RAID 2 具有如下技术特点。

①按位交叉存储，条带单元大小为 1 个比特位。

②采用海明校验技术，校验盘数量与使用的数据盘的数量成正比，具有纠错和检错功能，数据的可靠性高，但控制较复杂。

③每个 I/O 请求都会访问到多个磁盘：在写入时，需要计算每个条带数据位的校验位信息，并会把新写入的数据和校验位一起写入磁盘阵列；

在读取时，要同时读取所有的磁盘，数据位和相应的校验位都会被送至控制器进行即时校验。

④由于按位存取，I/O 过程中，所有磁盘上的磁头在任何时刻都处于同一位置，所有磁盘都并行工作；如果忽略接口带宽瓶颈，顺序访问性能随磁盘数目线性增长，但随机访问性能与单盘相同。受成本的影响，目前 RAID 2 很少被使用。

(4) RAID3

位交叉奇偶校验的磁盘阵列。

RAID3 只需要一个校验盘，其存储利用率更高，控制更简单，但各磁盘仍然按位交叉编址，所以磁盘仍然时常同步并行工作，其性能特点与 RAID2 类似。

(5) RAID4

块交叉奇偶校验的磁盘阵列。

RAID 4 的技术特点如下。

①采用奇偶校验技术，但采用较大的条带单元。

②各盘采用独立存取技术，顺序访问和随机访问性能均可随着盘的数目线性提升。

③磁盘利用率高。

④校验盘成为写访问的瓶颈，写任何一个数据块都需要更新校验盘数据，所以校验盘写入次数远多于数据盘，磨损最快，故障率最高。

RAID4 不适合应用于有大量写操作的场合。

(6) RAID 5

奇偶校验磁盘阵列。RAID5 对 RAID4 进行了针对性改进，其将校验数据以循环方式放在每个磁盘中，有效避免了校验盘成为写访问瓶颈的问题，这也使所有磁盘的磨损率一样，从而进一步提高了磁盘阵列系统的可靠性和安全性，但其只能支持一块磁盘故障。

RAID5 对大、小数据的读与写都有较好的性能，是应用最为广泛的阵列级别。

【1800 题举例】

45. 以下关于 RAID 技术的叙述中，错误的是（ ）。

- A. RAID 技术可以实现海量后备存储系统
- B. RAID 技术可以提高存储系统的可靠性
- C. RAID 中的校验信息都存放在一个磁盘上
- D. RAID 通过多个盘并行访问来提高速度

46. 【2013 年统考真题】下列选项中，用于提高 RAID 可靠性的措施有（ ）。

- | | | | |
|-----------|------------|-----------------|------------------|
| I. 磁盘镜像 | II. 条带化 | III. 奇偶校验 | IV. 增加 Cache 机制 |
| A. 仅 I、II | B. 仅 I、III | C. 仅 I、III 和 IV | D. 仅 II、III 和 IV |

47. 某 RAID 5 阵列将奇偶校验块按固定磁盘顺序分布(如磁盘 0→1→2→3→0 循环)，其优点是（ ）。

- A. 避免单个磁盘成为奇偶校验写入瓶颈
- B. 减少奇偶校验计算时间

- C. 提高存储空间利用率
- D. 支持同时写入多个数据块

3.5 高速缓冲存储器

对比王道 3.5.2 Cache 的基本工作原理

(一) 未说明清楚 Cache 命中率分析过程（它说未命中时，是先从主存读取包含目标数据的一个主存块送入 Cache，再将所需数据送到 CPU，但这两个顺序是同时发生）

【图领讲义】

3.Cache 命中率的计算

Cache 的容量与块长是影响 Cache 效率的重要因素，通常用“命中率”来衡量 Cache 的效率。

命中率是指 CPU 要访问的信息已在 Cache 内的比率。

在一个程序执行期间，设 N_c 为访问 Cache 的总命中次数， N_m 为访问主存的总次数，则命中率 h 为：

$$h = N_c / (N_c + N_m)。$$

设 t_c 为命中时 Cache 的访问时间，即 CPU 直接从 Cache 读取数据的时间。

未命中时的访问时间包括访问主存的时间 t_m 和访问 Cache 的时间 t_c ，其中 t_m 表示缺失损失，即从主存调入一个块所需的时间。

$1-h$ 表示未命中率，则 Cache-主存系统的平均访问时间为 t_a 为：

$$t_a = ht_c + (1-h)(t_m + t_c) = t_c + (1-h)t_m。$$

当然，以较小的硬件代价使 Cache-主存系统的平均访问时间 t_a 越接近于 t_c 越好。也可以说，命中率 h 越接近 1 越好。

若以 $e = t_c/t_a$ 作为访问效率，设 $r = t_m/t_c$ 表示主存与 cache 访问时间的倍数，则有：

$$e = \frac{t_c}{t_a} = \frac{t_c}{t_c + (1-h)t_m} = \frac{1}{1 + (1-h)r}$$

由上述式子可看出，存储系统的访问效率与 r 和 h 有关，命中率 h 的值越接近 1，访问效率越高；另外 r 的值不能太大，一般以 5~10 倍为宜。命中率与程序局部性、cache 容

量、数据块大小、组织方式有关。

【图领提示】

未命中时，需要从主存读取一个主存块送 Cache，同时将所需的数据送 CPU，该过程所需的时间为 t_m ，最后还需加上之前访问 cache 的时间 t_c ，总耗时为 t_m+t_c 。即先查 Cache，未命中再访问主存的方式。但有些教材采用同时查 Cache 和主存的方式，即命中，就提前终止主存访问。若没有特殊说明，统考默认采用先访问 Cache，未命中再访问主存的方式。

【1800 题举例】

56.某计算机的存储系统由 Cache-主存系统构成，Cache 的存取周期为 10 ns，主存的存取周期为 100 ns。当 CPU 执行一段程序时，Cache 完成存取的次数为 4500 次，主存完成存取的次数为 500 次，若采用先访问 Cache 再访问主存的方式进行存储，则该 Cache-主存系统的效率是（ ）。

- A. 0.423 B. 0.6 C. 0.5 D. 0.53

57.假设在具有两级 Cache 的存储系统中，某程序在执行的过程中访存 1000 次，其中访问一级 Cache 有 30 次没命中，然后访问二级 Cache，有 20 次没命中，则 Cache 的总命中率率为（ ）。

- A. 95% B. 96% C. 98% D. 99%

（二）未介绍 Cache 缺失对总体性能的影响

计算机性能最直接的度量方式就是 CPU 时间。程序执行的 CPU 时间应该等于 CPU 执行时间和等待主存访问时间之和。当发生 cache 缺失时，需要等待主存访问，此时，CPU 处于阻塞状态。因此，CPU 时间的计算公式如下：

$$\text{CPU 时间} = (\text{CPU 执行时钟数} + \text{cache 缺失引起阻塞的时钟数}) \times \text{时钟周期}$$

$$\text{cache 缺失引起阻塞的时钟数} = \text{读操作阻塞时钟数} + \text{写操作阻塞时钟数}$$

对于写操作，不同写策略下阻塞时钟数的计算方式不同。回写方式下，替换时需要一次性写回一个块，故会产生一些附加写回阻塞；全写方式下，包括写缺失阻塞和写缓冲阻塞两部分。假定写回阻塞和写缓冲阻塞忽略不计，则可将读操作和写操作综合考虑，得到如下公式：

$$\begin{aligned}\text{cache 缺失引起阻塞的时钟数} &= \text{程序中访存次数} \times \text{缺失率} \times \text{缺失损失} \\ &= \text{程序的指令条数} \times (\text{缺失数/指令}) \times \text{缺失损失}\end{aligned}$$

【教材例题】

例 7.11 假设计算机中只有一级 cache，并将指令和数据分别存放在 code cache 和 data cache 中，其缺失率分别为 1%和 4%。假定在没有任何访存阻塞时 CPI 为 1，缺失损失为 200 个时钟周期，访存指令的使用频度为 36%，每条访存指令存取一次数据，则使用缺失率为 0 的 cache 时，处理器速度会快多少？

解：假设程序运行共执行了 I 条指令，每条指令的取指令操作访存一次，则在访问指令发生缺失的情况下，所用的时钟周期数为 $I \times 1\% \times 200 = 2.0 \times I$ 。

已知访存指令的频度为 36%，故访问数据缺失时所用时钟周期数为 $I \times 36\% \times 4\% \times 200 = 2.88 \times I$ 。

因为在一条指令执行过程中取指令和访问数据总是串行进行的，所以，两者的阻塞时钟数应该相加，即指令缺失和数据缺失时的总阻塞时钟周期数为 $2.0 \times I + 2.88 \times I = 4.88 \times I$ ，也即平均每条指令要有 4.88 个时钟周期处于访存阻塞状态，因此，由于访存阻塞而使得 CPI 数从没有访存阻塞时的 1 增大到 $1 + 4.88 = 5.88$ 。所以，如果 cache 不发生缺失（即缺失率为 0），则处理器速度会快 $5.88/1 = 5.88$ 倍。

访存阻塞所用时间占整个执行时间的比例为 $4.88/5.88 \approx 83\%$ 。

对上述例子进一步分析，可以得到处理器的性能与 cache 性能之间的依赖关系。分以下两方面来考虑。

（1）假设上例中没有任何访存阻塞时 CPI 为 2，时钟宽度不变，则访存阻塞使得 CPI 数从 2 增加到 $2 + 4.88 = 6.88$ 。如果 cache 不发生缺失，则处理器速度会快 $6.88/2 = 3.44$ 倍。访存阻塞所用时间占整个执行时间的 $4.88/6.88 \approx 71\%$ ，小于 83%。

因此，可得出结论：CPI 越小，cache 缺失引起的阻塞对系统总体性能的影响越大。

（2）假定上例中时钟频率加倍，CPI 不变，则主存速度不太可能改变，故绝对时间不变，所以缺失损失变为 400 个时钟周期。访问指令缺失和数据缺失时引起的总阻塞时钟数为 $(1\% \times 400) + 36\% \times (4\% \times 400) = 9.76$ 。因此，访存阻塞使得 CPI 数从 1 增大到 $1 + 9.76 = 10.76$ ，

由于时钟频率加倍，相对于原时钟频率，其 CPI 相当于只有一半。由此可知，时钟频率快的机器的性能只是较慢机器的 $5.88/(10.76/2) \approx 1.1$ 倍。如果没有 cache 缺失，应该是 2 倍。

由此可得出结论：CPU 时钟频率越高，cache 缺失损失就越大。

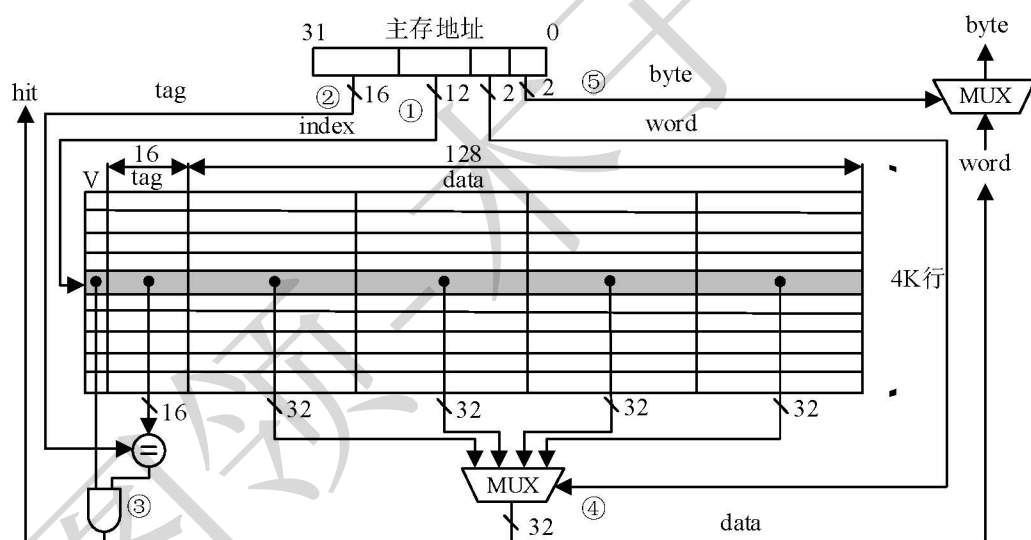
由上述两方面的分析结果可知，处理器性能越高，cache 缺失对总体性能的影响就越大。

对比王道 3.5.3 Cache 和主存的映射方式

（一）未给出直接映射的硬件结构下具体的访问过程

【图领讲义】

直接映射方式下主存地址的划分以及具体访存过程实现如下图所示。图中 tag 表示标记字段；index 表示 cache 行索引，即行号；块内地址分为两部分：高两位(word 字段)为字偏移量、低两位(byte 字段)为字节偏移量。hit 表示命中。



整个访存过程由硬件实现，分为以下 5 个步骤：

- ①根据 12 位 cache 行索引找到对应行；②将 16 位标志与对应行中的标志进行比较；
- ③比较相等并有效位为 1 时，输出 hit 为 1；④由两位字偏移量从 4 个 32 位字中选择一个字输出；⑤由两位字节偏移量从 1 个 32 位字中选择一字节输出。

CPU 在 hit 为 1 的情况下，根据要访问的是字还是字节选择从第④步还是第⑤步得到结果。若 hit 不为 1，则 CPU 要启动一次“cache 行读”总线事务操作，通过总线到主存读一

块连续的信息到 cache 行中。

(二) 未给出组相联映射方式下硬件结构和具体访问过程

【图领讲义】

实现组相联映射的硬件线路如图 3-29 所示。

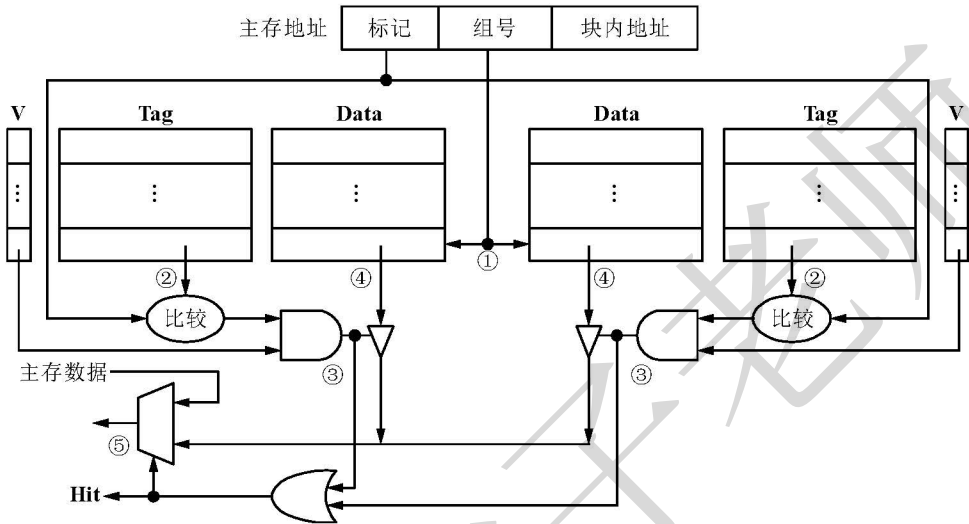


图 3-29 组相联方式的硬件实现

上图采用 2 路组相联映射的 cache，整个访存过程如下：①根据主存地址中的 cache 组号找到对应组。②将地址中的标记与对应组中每个行的标记 Tag 进行比较，③将比较结果和有效位 V 相“与”。④若有一路比较相等且有效位为 1，则输出 Hit 为 1，并选中这一路 cache 行中的主存块。⑤在 Hit 为 1 的情况下，根据主存地址中的块内地址从选中的一块内取出对应单元的信息，若 Hit 不为 1，则 CPU 要到主存读一块信息到 cache 行中。

对比王道 3.5.7 Cache 的应用。

(一) 未给出 Cache 设计应考虑的主存—总线—Cache 间的连接结构问题

【图领强化讲义】

上述考点角度是在 Cache 设计中考虑了采用分离 Cache/多级 Cache 联合的问题，这里再补充 Cache 设计考虑主存—总线—Cache 间的连接结构问题。

为了计算主存块传送到 cache 所用的时间，必须先了解 CPU 从主存取一块信息到 cache 的过程。从主存读一块数据到 cache，一般包含以下 3 个阶段。

①发送地址和读命令到主存：假定用 1 个时钟周期。

②主存准备好一个数据：假定用 10 个时钟周期。

③从总线传送一个数据：假定用 1 个时钟周期。

主存、总线和 cache 之间可以有 3 种连接方式：

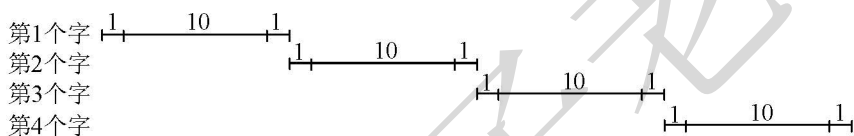
(1) 窄形结构，每次按一个字的宽度进行传送。

(2) 宽形结构，每次传送多个字。

(3) 多模块交叉存取结构，轮流启动多个存储模块进行读写，按一个字的宽度进行传送。

假定一个主存块有 4 个字，那么对于这 3 种结构，其缺失损失各是多少呢？

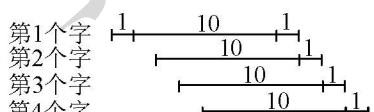
如下图所示，给出了 3 种方式下主存的传送过程。



(a) 窄形结构对应的块传送过程



(b) 宽形结构对应的块传送过程



(c) 交叉存储结构对应的传送过程

其中：

①窄形结构（单字串行传送）。缺失损失 = $4 \times (1T + 10T + 1T) = 45$ 个时钟周期（连续进行“送地址—读出—传送”4 次，每次传 2 个字）。

②宽形结构（2 字并行总线）。缺失损失 = $2 \times (1T + 10T + 1T) = 24$ 个时钟周期（连续进行“送地址—读出—传送”2 次，每次传 2 个字）。

③4 体低位交叉存取结构。缺失损失 = $1T + 10T + 3 \times 1T + 1T = 15$ 个时钟周期（发地址 1 次，流水线轮流启动 4 个存储体，主存准备和传送重叠进行，总线传送最后一个字）。

综上所述可知，多模块交叉存储结构的性价比最高。

【1800 题举例】

64. 假设 CPU 通过存储器总线读取数据的过程为：发送地址和读命令需 1 个时钟周期，存储器准备一个数据需 6 个时钟周期，总线上每传送 1 个数据需 1 个时钟周期。若主和

Cache 之间交换的主存块大小为 32B，存取宽度和总线宽度都为 8B，则 Cache 的一次缺失损失至少为（ ）个时钟周期。

- A. 32 B. 64 C. 80 D. 72

65.假定采用多模块交叉存储器的组织方式，存储器芯片和总线支持突发传送，CPU 通过存储器总线读取数据的过程为：发送首地址和读命令需 1 个时钟周期，存储器准备数据第一个数据需 8 个时钟周期，随后的每个时钟周期总线上传送 1 个数据，可连续传送 8 个数据（即突发长度为 8）若 Cache 和主存之间交换的块大小为 64B，存取宽度和总线宽度都为 8B，则 Cache 一次缺失至少损失（ ）个时钟周期。

- A. 17 B. 20 C. 34 D. 65

3.6 虚拟存储器

对比王道 3.6.2 页式虚拟存储器。

（一）未介绍 MMU 的相关概念和处理地址转换的检测情况

【图领强化讲义】

考查角度 2 虚拟地址与物理地址转换及过程分析

【考察方式】选择题、大题

【深入剖析】

该部分内容在 1.8 部分介绍过，这里不再赘述。在大题和选择题中均有涉及，须熟练掌握虚拟地址和物理地址的结构划分，地址转换的过程。

【图领拓展】

对于虚拟地址和物理地址的转换，需要 MMU（内存管理单元）实现，MMU 是计算机硬件中的一个组件，负责将虚拟地址转换为物理地址。在地址转换过程中，MMU 会检查以下情况：

①页表查询：MMU 通过页表将虚拟地址映射到物理地址。页表项中包含物理页框号、权限位（读、写、执行）、有效位（指示页面是否在内存中）等。

②TLB（转换检测缓冲器）管理：TLB 是 MMU 的一部分，用于缓存最近使用的页表项，以加速地址转换。如果 TLB 命中，则直接获取物理地址；如果 TLB 缺失，MMU 需

要访问内存中的页表。

③权限检查：MMU 会检查页表项中的权限位，确保当前操作（如读、写）符合权限要求。如果违反权限，则触发访问越权异常（如段错误）。

④页面存在检查：MMU 检查页表项的有效位。如果页面不在内存中（有效位为无效），则触发页面缺失（缺页）异常，由操作系统处理缺页中断。

其转换流程为：CPU 生成虚拟地址 → MMU 检查 TLB → 如果 TLB 缺失，访问页表 → 检查权限和页面有效性 → 生成物理地址。

【1800 题举例】

81.【2024 年统考真题】下列事件中，不是在 MMU 地址转换过程中检测的是（ ）。

- A. 访问越权 B. Cache 缺失 C. 页面缺失 D. TLB 缺失

（二）未解释 Cache-主存层，主存-外存层的结构实现

【图领强化讲义】

考点角度 6：页式虚拟存储器的层级结构实现分析

【考察方式】选择题

【深入剖析】

该部分内容考察频率较低，历年真题中只考察过一次，最主要还是理清楚页式虚拟存储器的实现。下面用表格说明页式虚拟存储器的两个层次结构的实现。

特性	Cache-主存层次	主存-外存层次
目标	提高访存速度	扩大程序可用内存空间
交换单位	主存块/Cache 行	页
管理实现者	专用硬件	操作系统（软件）为主，硬件（MMU）辅助
替换算法实现	硬件实现	操作系统软件实现
映射方式	直接映射、组相联、全相联	全相联映射（虚拟页可放入任何物理页框）
写策略	直写、回写	回写（通过脏位标志）

【图领总结】

这里再拓展总结 cache-主存和主存-辅存相关的异同点。

相同点：

①两者都是基于程序访问的局部性特点，把一块相邻的局部信息从慢速存储器复制到快速存储器。

②都必须考虑慢速存储器和快速存储器之间的映射问题。

③都需要考虑替换算法的实现。

④在快速存储器找不到信息时，都需要从慢速存储器装入该信息到快速存储器中。

不同点：

①位置不同。cache 最靠近 CPU，辅存最远离 CPU。CPU 可直接访问 cache，但不能直接访问辅存，主存和辅存直接交换数据。

②目的不同。cache-主存层的目的是为了加快 CPU 访问信息的速度；主存-辅存层次采用虚拟存储器机制是为了使程序不受内存容量的限制，扩大系统的存储容量。

③信息交换的块大小不同。cache-主存层交换的信息块为主存块，主存-辅存层交换的信息块为页。

④缺失处理不同。cache-主存层的缺失处理由硬件实现，而主存-辅存层则由操作系统（软件）实现。

⑤映射方式不同。cache-主存层可根据不同情况使用直接、全相联或组相联方式，映射方式完全由硬件实现，使用 cache 行标记字段来描述；主存-辅存层则都采用全相联方式，映射关系由操作系统实现，使用页表来描述映射关系。

⑥写策略不同。cache-主存层可要采用直写和回写两种策略；而主存-辅存层则都采用回写策略，若采用直写，每次写操作都要访问磁盘，开销过大。

⑦替换算法不同，cache-主存层的替换算法由硬件实现，主存-辅存层替换算法由软件实现。

【1800 题举例】

考点角度 6：页式虚拟存储器的层级结构实现分析

89. 【2024 年统考真题】对于页式虚拟存储管理系统，下列关于存储器层次结构的叙述中，错误的是（ ）。

- A. Cache-主存层次的交换单位为主存块，主存-外存层次的交换单位为页
- B. Cache-主存层次替换算法由硬件实现，主存-外存层次替换算法由软件实现
- C. Cache-主存层次可采用回写法写策略，主存-外存层次通常采用回写法写策略
- D. Cache-主存层次可采用直接映射方式，主存-外存层次通常采用直接映射方式

90. 下列关于 Cache-主存与主存-外存层次对比的描述中，正确的是（ ）。

- I. 两个层次都采用 LRU 替换算法，且都由硬件实现以保证效率。
- II. Cache-主存层次可采用直写策略，而主存-外存层次也通常采用直写策略确保数据一致性。
- III. Cache-主存层次的映射方式包括直接映射，而主存-外存层次采用全相联映射。
- IV. 两个层次对应用程序员都是透明的，管理机制完全相同。

- A. I、II B. II、III C. 仅III D. III、IV

（四）未解释 TLB 字段划分及表项内容的情况

【图领强化讲义】

考查角度 3 TLB 字段划分和计算分析

【考察方式】选择题、大题

【深入剖析】

TLB 表项需实现“虚拟页号→物理页号”的快速映射，其字段（标记、组索引、物理页号）位数由以下因素决定：

①页大小决定页内偏移位数，即 $\log_2$ 页大小。

虚拟地址位数 = 虚拟页号（VPN）+ 页内偏移。

②TLB 映射方式（组相联 / 全相联）。

组相联需计算组数，即总表项数 ÷ 路数，进而得组索引位数 $\log_2$ 组数。

③标记字段位数 = VPN 总位数 - 组索引位数。

④全相联映射无组索引，标记字段 = VPN 的位数。

【图领提示】

对于 TLB 表项来说，其一个表项通常包含 TLB 标记和数据，其中 TLB 标记对应虚拟

页号的高位部分，数据包含物理页号、控制信息位（有效位必须包含）以及辅助信息位。而组号通常是隐含的，其对应虚拟页号的低位部分。

【1800 题举例】

考点角度 5：TLB 字段划分和计算分析

86.【2024 年统考真题】某计算机按字节编址，采用页式虚拟存储管理方式，虚拟地址为 32 位，主存地址为 30 位，页大小为 1KB。若 TLB 共有 32 个表项，采用 4 路组相联映射方式，则 TLB 表项中标记字段的位数至少是（ ）。

- A. 17 B. 18 C. 19 D. 20

87.某计算机按字节编址，采用页式虚拟存储管理方式，虚拟地址为 32 位，主存地址为 30 位，页大小为 4KB。若 TLB 共有 64 个表项，采用全相联映射方式，则 TLB 表项中标记字段的位数至少是（ ）。

- A. 17 B. 18 C. 19 D. 20

88.某计算机按字节编址，采用页式虚拟存储管理方式，虚拟地址为 32 位，主存地址为 30 位，页大小为 1KB。若 TLB 共有 32 个表项，采用 4 路组相联映射方式，则一个 TLB 表项中的位数至少是（ ）。

- A. 39 B. 40 C. 42 D. 43

对比王道 3.6.3 段式虚拟存储器。

（一）未介绍分段式虚拟存储器的变化过程出现的异常情况

在分段式虚拟存储器系统中，虚拟地址由段号和段内地址组成。通过段表把虚拟地址变换成主存物理地址，其变换过程如图 7.45 所示。

每个进程的段表在主存的首地址都存放在段表基址寄存器中，根据虚拟地址中的段号，可找到对应段表项，以检查是否存在以下 3 种异常情况。

（1）缺段（段不存在）：装入位=0。

（2）地址越界：偏移量超出最大段长。

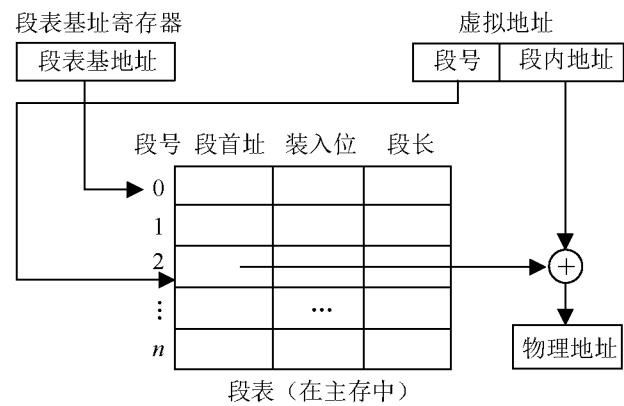


图 7.45 分段式虚存的地址转换

（3）访问越权：操作方式与指定访问权限不符。若发生以上 3 种情况，则调用相应的异常处理程序，否则，将段表项中的段首址与虚拟地址中的段内地址相加，生成访问主存时的物理地址。

第四章

4.1 指令系统

对比王道 4.1.2 指令的基本格式。

（一）未解释指令格式的设计原则

指令格式的选择应遵循如下几条基本原则。

（1）指令应尽量短。每条指令长度短，使得程序占用存储空间小，降低空间开销。

（2）要有足够的操作码位数。向后兼容使指令操作类型不断增加，因此必须预留足够的操作码位数。

（3）操作码的编码必须有唯一的解释。操作码最终需送到指令译码器进行译码，因此，指令操作码要么是一个唯一的合法编码，要么是不合法的 0/1 序列。当译码器发现是不合法操作码时，出现“非法指令”异常。

（4）指令长度应是字节的整数倍。指令执行前存放在内存，而内存往往按字节编址，因此，指令长度为字节的整数倍，便于指令的读取和指令地址的计算。

（5）合理选择地址字段的个数。地址字段个数涉及指令的长度和指令的规整性问题，它是空间开销和时间开销权衡的结果。

（6）指令应尽量规整。指令的规整性体现在许多方面，如指令长度是否固定、操作码位数是否固定、地址码格式是否一致、指令字中各字段的划分位置是否一致等。规整的指令格式会大大简化硬件的实现。

4.2 寻址方式

对比王道 4.2.1 指令寻址和数据寻址

（一）未解释相对寻址的偏移量的变化情况

【图领强化讲义】

【深入剖析】

该内容在选择题和大题中均有涉及且在大题考查频繁，综合难度较大，应当熟练掌握。这部分涉及的内容主要是相对寻址的跳转地址的具体计算。通常计算公式为 $PC + “1”$

+OFFSET。其中这个“1”跟编址方式和指令长度有关，OFFSET 的偏移单位也跟编址方式和相对变化有关。具体情况如下所示。

①若机器字长为 16 位，按字节编址，指令长度占 2 个字节，若偏移量是相对字节变化，则跳转地址= $PC+2+OFFSET$ 。

②若机器字长为 16 位，按字节编址，指令长度占 2 个字节，若偏移量是相对字变化，则跳转地址= $PC+2+2\times OFFSET$ 。

③若机器字长为 16 位，按字编址，指令长度占 2 个字节，若偏移量是相对字变化，则跳转地址= $PC+1+OFFSET$ 。

【图领提示】

①如果题目没有明确说明偏移量是相对什么变化，则默认是相对指令字的位数为单位变化，即以指令的最小对齐单位为变化。比如采用 16 位定长指令字格式，则偏移量是相对字长 16 位为单位变化。

②如果题目没有给出指令字格式，也没有说明偏移量相对什么变化，则默认相对编址方式为单位变化，比如按字节编址，则相对字节为单位变化。

【1800 题举例】

14. 某机器字长 16 位，主存按字节编址，转移指令采用相对寻址，由两个字节组成，第一字节为操作码字段，第二字节为相对位移量字段。假定取指令时，每取一个字节 PC 自动加 1。若某转移指令所在主存地址为 3000H，相对位移量字段内容为 06H，则该转移指令成功转移后的目标地址是（ ）。

- A. 3006H B. 3007H C. 3008H D. 3009H

15. 某机器指令字长为 32 位，按字节编址，取指令时，每取一个字节，PC 自增 1，当前指令地址为 5000H，指令内容为相对寻址的无条件转移指令，指令中的形式地址为 50H，则取指令后以及指令执行后 PC 的值是（ ）。

- A. 5002H 5052H B. 5000H 5050H
C. 5004H 5054H D. 5004H 5050H

16. 某机器字长 16 位，主存按字编址，转移指令采用相对寻址，由两个字节组成，第一字节为操作码字段，第二字节为相对位移量字段。假定取指令时，每取一个字 PC 自动加 1。若某转移指令所在主存地址为 3000H，相对位移量字段内容为 06H，则该转移指令

成功转移后的目标地址是（ ）。

- A. 3006H B. 3007H C. 3008H D. 3009H

17. 【2009 年统考真题】某机器字长为 16 位，主存按字节编址，转移指令采用相对寻址，由两个字节组成，第一个字节为操作码字段，第二个字节为相对位移量字段。假定取指令时，每取一个字节 PC 自动加 1。若某转移指令所在主存地址为 2000H，相对位移量字段的内容为 06H，则该转移指令成功转移后的目标地址是（ ）。

- A. 2006H B. 2007H C. 2008H D. 2009H

（二）未解释各类寻址方式的寻址范围的情况

个人补充：

各类寻址方式涉及的范围如下表所示（默认按字节编址）。

寻址方式	EA 计算公式	寻址范围的决定因素	32 位机示例
立即寻址	无 EA，操作数=A	地址码 A 的位数	A=8 位：-128~127(补码) 或 0~255（无符号）
直接寻址	EA=A	地址码字段 A 的位数	A=16 位：0~65535(64KB 主存空间)
一次间接寻址	EA=主存[A]	存储字长的位数	存储字长 32 位：0~2 ³² -1 (4GB 主存空间)
寄存器寻址	无 EA，操作数=Ri	CPU 中通用寄存器的数量	16 个通用寄存器，地 址码需 4 位
寄存器间接寻址	EA= (Ri)	通用寄存器的位数	寄存器 32 位：0~2 ³² -1 (4GB 主存空间)
相对寻址	EA= (pc) +A	偏移量 A 的位数(补 码表示)	A=8 位：(PC) -128~ (PC) +127 (256B)

【1800 题示例】

22. 某计算机机器字长为 16 位，操作码占 5 位，寻址方式位占 3 位，若采用直接寻址方式，故其可寻址的范围为（ ）。

- A. 128 B. 255 C. 256 D. 32768

23. 【2020 年统考真题】某计算机采用 16 位定长指令字格式，操作码位数和寻址方式位数固定，指令系统有 48 条指令，支持直接、间接、立即、相对 4 种寻址方式。在单地址指令中，直接寻址方式的寻址范围是（ ）。

- A. 0~255
- B. 0~1023
- C. -128~127
- D. -512~511

4.3 程序的机器级代码表示

对比王道 4.3.1 常用汇编指令

(一) 未介绍 MIPS 汇编语言和指令格式

1.MIPS 指令中数据的表示

MIPS 提供了 32 个 32 位通用寄存器，寄存器编号占 5 位，各寄存器的名称、编号和功能见表 4.2。

表 4.2 MIPS 通用寄存器

名称	编号	功能
zero	0	恒为 0
at	1	为汇编程序保留
v0~v1	2~3	过程调用返回值
a0~a3	4~7	过程调用参数
t0~t7	8~15	临时变量，在被调用过程无须保存
s0~s7	16~23	在被调用过程需保存
t8~t9	24~25	临时变量，在被调用过程无须保存
k0~k1	26~27	为 OS 保留
gP	28	全局指针
sp	29	栈指针
fp	30	帧指针
ra	31	过程调用返回地址

寄存器的汇编表示以\$符号开始,可以使用名称(如\$a0)，也可以使用编号(如\$4)。

MIPS 还提供了 32 个 32 位的单精度浮点寄存器，用汇编符号\$f0~\$f31 表示。它们可配对成 16 个 64 位浮点寄存器，用来表示 64 位双精度浮点数。

另外，MIPS 中提供了两个乘商寄存器 Hi 和 Lo，它们无须程序员在指令中显式给出。

用 32 位的 Hi 和 Lo 可实现 64 位寄存器。在执行乘法运算时，Hi 和 Lo 联合用来存放 64 位乘积，而在执行除法运算时，最终的余数存放在 Hi 中，商在 Lo 中。

MIPS 中用程序计数器 PC 指出下条指令的地址。

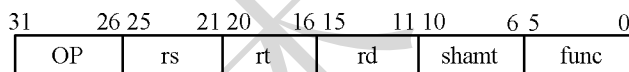
MIPS 的存储器按字节编址。对于存储器数据，其操作数有效地址为 32 位，通过一个 32 位寄存器的内容加 16 位偏移量得到，16 位偏移量是带符号整数，故可访问的地址空间大小为 2^{32} 字节。采用大端方式(bigendian)存放数据，数据要求按字边界对齐。只能通过 load/store 指令访问存储器数据。

对于立即操作数，指令中给出的位数为 16 位，指令执行时，需要将其进行符号扩展或 0 扩展，变成 32 位操作数后才能参加运算。

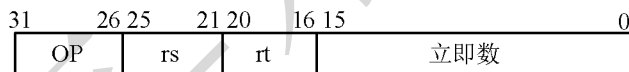
2.MIPS 指令格式和寻址方式

MIPS 是典型的 RISC 处理器，采用 32 位定长指令字，操作码字段也是固定长度，没有专门的寻址方式字段，由指令格式确定各操作数的寻址方式。

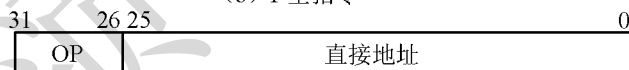
指令格式只有 3 种，如图 4.5 所示。



(a) R-型指令



(b) I-型指令



(c) J-型指令

图 4.5 MIPS 指令格式

R-型指令属于 RR 型指令，即操作数和结果都存放在寄存器中，这里 R 表示寄存器(register)。其操作码 OP 为 000000，操作类型由 func 字段指定，若是双目运算类指令，则 rs 和 rt 的内容分别作为第一和第二源操作数，结果送 rd；若是移位指令，则对 rt 的内容进行移位，结果送 rd，所移位数由 shamt 字段给出。因为一条指令需要左移或右移若干位，所以 MIPS 中移位指令多用桶形移位器实现以提高速度。R-型指令的寻址方式只有一种，就是寄存器寻址。

I-型指令是立即数型指令，这里 I 表示立即数(immediate operand)。若是双目运算类指令，则将 rs 的内容和立即数分别作为第一和第二源操作数，结果送 rt；若是 load/store 指令，则将 rs 的内容和立即数符号扩展后的内容相加作为内存单元地址，load 指令将内存单元内容送 rt，store 指令将 rt 内容送内存单元；若是条件转移(分支)指令，则对 rs 和 rt 内容进行指定的运算，根据运算的结果，决定是否转到转移目标地址处执行，转移目标地址通过相对寻址方式得到，即将 PC 的内容和立即数符号扩展后的内容相加得到。由此可知，I-型指令的寻址方式有 4 种，就是寄存器寻址、立即数寻址、相对寻址、基址或变址寻址。

J-型指令主要是无条件跳转指令，这里 J 表示跳转(jump)。指令中给出的是 26 位直接地址，只要将当前 PC 的高 4 位拼上 26 位直接地址，最后添两个 0 就可以得到 32 位的跳转目标地址。J-型指令的寻址方式只有一种，就是变通的直接寻址。

3.MIPS 汇编语言

表 4.3 和表 4.4 分别是 MIPS 汇编语言和机器代码示例列表。表 4.3 中列出了常用的类指令：算术运算、存储访问、逻辑运算、条件分支、无条件跳转。每类中给出最具代表性的指令的名称、汇编形式示例、含义和文字说明。表 4.4 中给出了常用指令的机器代码示例，分别包括操作码汇编助记符、指令格式类型、指令各字段的十进制值和对应的汇编表示。

表 4.3 MIPS 汇编语言示例列表

类别	指令名称	汇编举例	含义	备注
算术运算	add	add \$s1, \$s2, \$s3	$\$s1 = \$s2 + \$s3$	三个寄存器操作数
	subtract	sub \$s1, \$s2, \$s3	$\$s1 = \$s2 - \$s3$	三个寄存器操作数
存储访问	load word	lw \$s1, 100(\$s2)	$\$s1 = \text{Memory}[\$s2 + 100]$	从内存取一个字到寄存器
	store word	sw \$s1, 100(\$s2)	$\text{Memory}[\$s2 + 100] = \$s1$	从寄存器存一个字到内存
逻辑运算	and	and \$s1, \$s2, \$s3	$\$s1 = \$s2 \& \$s3$	三个寄存器操作数，按位与
	or	or \$s1, \$s2, \$s3	$\$s1 = \$s2 \$s3$	三个寄存器操作数，按位或
	nor	nor \$s1, \$s2, \$s3	$\$s1 = \sim(\$s2 \$s3)$	三个寄存器操作数，按位或非
	and immediate	andi \$s1, \$s2, 100	$\$s1 = \$s2 \& 100$	寄存器和常数，按位与

类别	指令名称	汇编举例	含义	备注
	or immediate	ori \$s1, \$s2,100	\$s1= \$s2 00	寄存器和常数，按位或
	shift left logical	sll \$s1, \$s2,10	\$s1= \$s2<<10	按常数对寄存器逻辑左移
	shift right logical	srl \$s1, \$s2,10	\$s1= \$s2>>10	按常数对寄存器逻辑右移
条件分支	branch on equal	beq \$s1, \$s2,L	if(\$s1== \$s2) go to L	相等则转移
	branch on not equal	bne \$s1, \$s2,L	if(\$s1!= \$s2) go to L	不相等则转移
	set on less than	slt \$s1,\$s2, \$s3	if(\$s2<\$s3) \$s1=1; else \$s1=0	小于则置寄存器为1，否则为0，用于后续指令判0
	set on less than immediate	slt \$s1,\$s2,100	if(\$s2<100)\$s1=1; else \$s1=0	小于常数则置寄存器为1，否则为0，用于后续指令判0
无条件跳转	jump	j L	go to L	直接跳转至目标地址
	jump register	jr \$ra	go to \$ra	过程返回
	jump and link	jal L	\$ra=PC+4; go to L	过程调用

表 4.4 MIPS 机器代码示例列表

指令	格式	指令举例						备注
add	R	0	18	19	17	0	32	add \$s1, \$s2, \$s3
sub	R	0	18	19	17	0	34	sub \$s1, \$s2, \$s3
lw	I	35	18	17	100			lw \$s1,100(\$s2)
sw	I	43	18	17	100			sw \$s1,100(\$s2)
and	R	35	18	19	17	0	36	and \$s1, \$s2, \$s3
or	R	0	18	19	17	0	37	or \$s1, \$s2, \$s3
nor	R	0	18	19	17	0	39	nor \$s1, \$s2, \$s3
andi	I	12	18	17	100			andi \$s1, \$s2,100
ori	I	13	18	17	100			ori \$s1, \$s2,100
sll	R	0	0	18	17	10	0	sll \$s1, \$s2,10
srl	R	0	0	18	17	10	2	srl \$s1, \$s2,10
beq	I	4	17	18	25			beq \$s1, \$s2,100
bne	I	5	17	18	25			bne \$s1, \$s2,100
slt	R	0	18	19	17	0	42	slt \$s1, \$s2, \$s3
j	J	2	2500					j 10000

指令	格式	指令举例						备注
jr	R	0	31	0	0	0	8	jr \$ra
jal	J	3	2500					jal 10000
字段大小		6 位	5 位	5 位	5 位	5 位	6 位	
R-型指令	R	OP	rs	rt	rd	shamt	func	
I 型指令	I	OP	rs	rt	address			

个人补充：

在 32 位 MIPS 指令系统中，指令固定长度为 32 位，按字节编址且按字边界对齐存放（也就是偏移量存的是指令条数，相对字为单位变化），其采用相对寻址的计算目标转移地址的计算公式为： $PC+4+4\times\text{OFFSET}$ 。

【1800 题示例】

3.【图领★】【仔细看下这道题】MIPS32 指令系统中的条件分支指令中的地址偏移量为 16 个 bit 位，那么该指令跳转范围为（ ）。

- A. 地址在 $0 \sim (64\text{ K}-1)$ 字节之间
- B. 地址在 $0 \sim (256\text{ K}-1)$ 字节之间
- C. 分支指令前后范围各大约 32K 字节
- D. 分支指令前后范围各大约 128K 字节

4.【图领★】在 32 位 MIPS 指令集架构中，条件跳转指令所在的地址为 1000，目标地址为 2000，那么该指令中 16 位偏移量的值为（ ）。

- A. 1000
- B. 256
- C. 250
- D. 249

第五章

5.6 指令流水线

对比王道 5.6.2 流水线的基本实现

(一) 未介绍其他流水线的相关性能指标及其计算方式

【图领强化讲义】

(2) 流水线的加速比

加速比是指完成同样一批任务，不使用流水线所用的时间与使用流水线所用的时间之比。

设 T_0 表示不使用流水线时的执行时间，即顺序执行所用时间， T_k 表示使用流水线时的执行时间，则计算流水线加速比 S 的基本公式为：

$$S = \frac{T_0}{T_k}$$

在一条 k 段流水线中，单独完成一个任务耗时为 $k\Delta t$ ，则顺序完成 n 个任务耗时为 $T_0 = nk\Delta t$ ，而 k 段流水线完成 n 个任务的时间为： $T_k = (k+n-1)\Delta t$ ，故实际加速比为：

$$S = \frac{nk\Delta}{(k+n-1)\Delta} = \frac{kn}{k+n-1}$$

当连续输入 n 趋向于无穷时就有：

$$\lim_{n \rightarrow \infty} S = \frac{kn}{k+n-1}$$

故

$$S_{\max} = 1$$

(3) 流水线的效率

流水线的设备利用率称为流水线的效率。在时空图上，流水线的效率定义为： n 个任务占用的时空区有效面积与 n 个任务所用的时间与 k 个流水段所围成的时空区域总面积之比，则流水线的一般公式为：

$$E = \frac{n \text{ 个任务占用的时空区有效面积}}{n \text{ 个任务所用的时间与 } k \text{ 个流水段所围成的时空区域总面积}} = \frac{T_0}{kT_k}$$

当连续输入 n 趋向于无穷时就有

$$\lim_{n \rightarrow \infty} E = \frac{T_0}{kT_k}$$

故

$$E_{\max} = 1$$

【1800 题示例】

55. 设指令由取指、分析、执行 3 个子过程完成，并且每个子过程的时间均为 t ，采用常规 标量单流水线处理机连续执行 8 条指令的加速比为（ ）。

- A. 3 B. 2 C. 3.4 D. 2.4

对比王道 5.6.4 流水线的冒险与处理

（一）未说明分支指令和无条件转移指令延迟时间损失片的情况

【图领强化讲义】

3. 控制冒险

主要由分支指令或跳转指令引起。在分支指令的执行阶段完成并计算出实际跳转目标地址或条件结果之前，后续指令（按顺序预取）可能已被取入流水线。如果实际执行路径与预取路径不同，这些预取指令无效。

若执行下面指令序列：

I1 beq R1, R2, Label ; 若 $R1 == R2$ 则跳转到 Label

I2 add R3, R4, R5 ;

五段式流水线中，beq 指令通常在 EX 段计算设置条件码，在 MEM 段确定是否将 PC 值更新为转移目标的地址，等 MEM 段结束后才写入 PC，故不做处理 CPU 直接取地址一定会取到按照顺序执行的 PC+1 的地址，此时最简单的处理方法是插入 3 条 nop 指令（即延迟损失时间片 $C=3$ ），可以保证在下一条指令的 IF 段，PC 已经被 I1 指令修改过，如下表所示。

指令	周期 1	周期 2	周期 3	周期 4	周期 5
I ₁	IF	ID	EX	MEM	WB
I ₂		nop	nop	nop	IF

控制冒险主要有两种解决办法：

①分支预测：在分支结果未知时，预测其行为（总是不跳转/总是跳转/基于历史动态

预测) 并提前取指。预测正确则无停顿; 预测错误则冲刷错误路径指令, 产生惩罚。

②延迟分支: 采用硬件阻塞方式(插入气泡)或软件阻塞方式(插入空操作指令)。假设延迟损失时间片为 C , 则在数据通路中检测到分支指令时, 就在分支指令后插入 C 个气泡, 或在编译时在分支指令后填入 C 条 `nop` 指令。

【图领拓展】

①对于无条件转移指令, 其在 `ID` 段生成无条件转移目标地址, 而后直接传送到 `EX/Mem` 寄存器输入端即可。在 `EX` 段结束后写入 `PC`。因此其分支延迟损失时间片为 2。

②若分支指令在 `ID` 段计算设置条件码, `EX` 段确定是否更新 `PC`, 而后再 `EX` 段结束时写 `PC`, 则其分支延迟损失时间片为 2。

【1800 题举例】

47. 【2023 年统考真题】某系统采用“取指、译码/取数、执行、访存、写回”5 段流水线, `RISC` 处理器中执行如下指令序列(第一列为指令序号), 其中 `s0`、`s1`、`s2`、`s3` 和 `t2` 表示寄存器编号。

```
I1  add s2, s1, s0  // R[s2] ← R[s1] + R[s0]
I2  load s3, 0(s2)  // R[s3] ← M[R[s2] + 0]
I3  beq t2, s3, L1   // if R[t2] = R[s3] jump to L1
I4  addi t2, t2, 20  // R[t2] ← R[t2] + 20
I5  L1:
```

若采用转发(旁路)技术处理数据冒险, 采用硬件阻塞方式处理控制冒险, 则在 `I1~I4` 执行过程中, 发生流水线阻塞的指令有()。

- A. 仅 `I3` B. `I2`、`I4` C. `I3`、`I4` D. `I2`、`I3`、`I4`

49. 在采用“取指(`IF`)、译码/取数(`ID`)、执行(`EX`)、访存(`MEM`)、写回(`WB`)”的 5 段流水线中, 某 `MIPS` 指令序列如下, 其支持寄存器转发技术, 但不支持分支预测:

```
I1  Loop:  add  $t0, $s1, $s2    ; $t0 = s1 + s2
I2          lw   $t1, 0($t0)     ; $t1 = 内存 [$t0]
I3          sub  $t2, $t1, $s3   ; $t2 = t1 - s3
I4          add  $t1, $t2, $s4   ; $t1 = t2 + s4
```

I5 bne \$t2, \$s5, Exit ;若 $t2 \neq s5$ 则跳至 Exit
 I6 add \$s1, \$s1, \$t1 ; $s1 = s1 + t1$
 I7 j Loop ; 跳回 Loop
 I8 Exit: sw \$t2, 4(\$t0) ;内存 $[t0+4] = t2$

假设所有指令均为单周期执行，则须插入阻塞周期的数据相关的指令对是（ ）；属于控制相关且会导致流水线冲刷的指令对是（ ）。

- A. I2 与 I3; I5 与 I6、I7
- B. I2 与 I3、I4 与 I1; I5 与 I6、I7、I8
- C. I2 与 I3; I5 与 I6、I7、I8
- D. I2 与 I3、I3 与 I4; I5 与 I6、I7

5.7 多处理器的基本概念

对比王道 5.7.4 共享内存多处理器的基本概念

（一）缺少对 NUMA 结构引入高速缓存一致性确认机制的解释

【图领强化讲义】

②非统一存储访问（NUMA）多处理器

在 NUMA 多处理器系统中，每个处理器都带有一个本地存储模块，与 UMA 一样，所有共享存储模块统一编址，以形成具有单一地址空间的一个逻辑存储器。只不过在 UMA 中必须保证访存时间一致，而在 NUMA 中处理器访问本地存储模块要快于对非本地存储模块（属于另一个处理器）的访问，其访问时间不一致。如果在 NUMA 结构中引入高速缓存一致性确认机制，则称为高速缓存一致的非一致性内存访问(cache coherent NUMA, CC-NUMA)。由于所有本地存储模块和非本地存储模块都在同一个地址空间，非本地存储模块也通过 load 和 store 指令来访问。NUMA 计算机上运行的所有 UMA 程序无须做任何改变，但在相同的主频下其性能不如 UMA 计算机上的性能。

【1800 题举例】

67. 【2022 年统考真题】下列关于并行处理技术的叙述中，不正确的是（ ）。

- A. 多核处理器属于 MIMD 结构

- B. 向量处理器属于 SIMD 结构
- C. 硬件多线程技术只可用于多核处理器
- D. SMP 中所有处理器共享单一物理地址空间

68. 下列关于多处理器系统叙述错误的是（ ）。

I. 采用 UMA 多处理器结构时，每个处理器对共享存储器的访问时间与其和存储器 的相对位置无关。

II. 采用 NUMA 多处理器结构时，非本地存储模块不能通过 load 和 store 指令来访问

III. 不管是 UMA 结构还是 UNMA 结构，两者都通过共享存储模块统一编址形成单一的地址空间

IV. CC-NUMA 多处理器结构必须实现 Cache 的一致性机制

- A. I、II、III
- B. I、III
- C. 仅II
- D. I、IV

第六章

6.1 总线概述

对比王道 6.1.3 总线的性能指标

(一) 未解释总线带宽计算所涉及具体因素的情况

【图领强化讲义】

考点 3 总线的性能指标

该部分内容在选择题和大题中均有涉及，属于概念计算类题型，应当熟练掌握。

考查角度 1：总线带宽（数据传输速率）的相关计算

【考察方式】选择题

【深入剖析】

该部分内容在选择题和大题中也经常结合其他章节知识点或者本章节中的突发传送知识点在大题中进行考察。要注意题目中给出的所有可用信息，还要注意单位转换。

总线带宽 (B/S) = (总线宽度 (bit) / 8) × 总线工作频率 (Hz) = 总线宽度 (bit) / 8 ÷ 总线周期。

【图领提示】

①如果题目没有给出总线工作频率，而是给的总线时钟频率，且告诉一个时钟周期可传送 n 次数据，则总线工作频率是总线时钟频率的 n 倍。

②若题目给出总线周期为 k 个时钟周期，则总线工作频率为：总线时钟频率 ÷ k 。

③如果题目给出特定总线条件下的带宽计算公式，则按题目的要求去算即可。

④若题目要求最大的数据传输速率，而不是求平均数据传输速率，则计算该最大带宽时，不需要具体到每一个时钟周期内数据的传送情况。

【1800 题举例】

1. 【2009 年统考真题】假设某系统总线在一个总线周期中并行传输 4 字节信息，一个总线周期占用 2 个时钟周期，总线时钟频率为 10MHz，则总线带宽是 ()。

A. 10 MB/s

B. 20 MB/s

C. 40 MB/s

D. 80 MB/s

2.【2024 年统考真题】某存储器总线的时钟频率为 420MHz，总线宽度为 64 位，每个时钟周期传送 2 次数据；其总线事务支持突发传送方式，最多传送 8 次数据，第 1 个时钟周期传送地址和读/写命令，从第 4 个至第 7 个时钟周期连续传送 8 次数据。该总线的总线带宽（最大数据传输率）为（ ）。

- A. 3.84 GB/s B. 6.72 GB/s C. 30.72 GB/s D. 53.76 GB/s

3.【2014 年统考真题】某同步总线采用数据线和地址线复用方式，其中地址/数据线有 32 根，总线时钟频率为 66MHz，每个时钟周期传送两次数据（上升沿和下降沿各传送一次数据），该总线的最大数据传输速率（总线带宽）是（ ）。

- A. 132 MB/s B. 264 MB/s C. 528 MB/s D. 1056 MB/s

（二）未对不同总线的方式涉及到的带宽计算和结构进行说明

1.处理器总线

早期的 Intel 微处理器的处理器总线称为前端总线(front side bus,FSB)，它是主板上最快的总线，主要用作处理器与北桥芯片进行信息交换。

FSB 的传输速率单位实际上是 MT/s，表示每秒传送多少百万次。MHz 是时钟频率单位，早期的 FSB 每个时钟周期传送一次数据，因此时钟频率与传输速率一致。但是，从 Pentium Pro 开始，FSB 采用四倍并发(quad pumped)技术,在每个总线时钟周期内传 4 次数据，也就是说总线的传输速率等于总线时钟频率的 4 倍，若时钟频率为 333MHz，则传输速率为 1333MT/s，即 1.333GT/s，但习惯上称 1333MHz。若前端总线的工作频率为 1333MHz(实际时钟频率为 333MHz)，总线宽度为 64 位，则总线带宽为 10.664GB/s。

Intel 公司推出 Corei7 时，北桥芯片的功能被集成到了 CPU 芯片内，CPU 通过存储器总线(即内存条插槽)直接和内存条相连，而在 CPU 芯片内部的核与核之间、CPU 芯片与其他 CPU 芯片之间，以及 CPU 芯片与 IOH(input/output hub)芯片之间,则通过 QPI(quick path interconnect，快速通道互联)总线相连。

QPI 总线是一种基于包传输的串行高速点对点连接协议，采用差分信号与专门的时钟信号进行传输。QPI 总线有 20 条数据线，发送方(TX)和接收方(RX)有各自的时钟信号，每个时钟周期传输 2 次。一个 QPI 数据包包含 80 位的数据，需要两个时钟周期或 4 次传

输才能完成整个数据包的传送。在每次传输的 20 位数据中，有 16 位是有效数据，其余 4 位用于循环冗余校验，以提高系统的可靠性。由于 QPI 是双向的，在发送的同时也可以接收另一端传输来的数据，这样，每个 QPI 总线的带宽计算公式为：

$$\text{QPI 总线带宽} = \text{每秒传输次数} \times \text{每次传输的有效数据} \times 2$$

QPI 总线的速度单位通常为 GT/s，若 QPI 的时钟频率为 2.4GHz，则速度为 4.8GT/s，表示每秒钟传输 4.8G 次数据，并称该 QPI 工作频率为 4.8GT/s。因此，工作频率为 4.8GT/s 的 QPI 总线的总带宽为 $4.8\text{GT/s} \times 2\text{B} \times 2 = 19.2\text{GB/s}$ 。工作频率为 6.4GT/s 的 QPI 总线的总带宽为 $6.4\text{GT/s} \times 2\text{B} \times 2 = 25.6\text{GB/s}$ 。

图 8.4 给出了 Intel Core i7 中核与核之间、核与主存控制器之间以及各级 cache 之间的互连结构。

从图 8.4 可以看出，一个 Core i7 处理器中有 4 个 CPU 核(core)，每两个核之间都用 QPI 总线互连，并且每个核还有一条 QPI 总线可以与 IOH 芯片互连。处理器支持三通道 DDR3 SDRAM 内存条插槽，因此，处理器中包含有 3 个主存控制器，并有 3 个并行传输的存储器总线，这意味着一组存储器总线包含 3 个内存条插槽，正如 7.2.4 节中图 7.9(a)所示的双通道内存条插槽一样，组内的内存条可以以并行方式存取信息。

2. 存储器总线

早期的存储器总线由北桥芯片控制，处理器通过北桥芯片和主存储器、图形卡(显卡)以及南桥芯片进行互连。Core i7 以后的处理器芯片中集成了主存控制器，因而，存储器总线直接连接到处理器，如图 8.4 所示。

根据芯片组设计时确定的芯片组能够处理的主存类型的不同，存储器总线有不同的运行速度。如图 8.4 所示的计算机中，存储器总线宽度为 64 位，每秒钟传输 1333M 次，总线带宽为 $1333\text{M} \times 64/8 = 10.664\text{GB/s}$ ，因而 3 个通道的总带宽为 32GB/s，与此配套的内存条型号为 DDR3-1333。

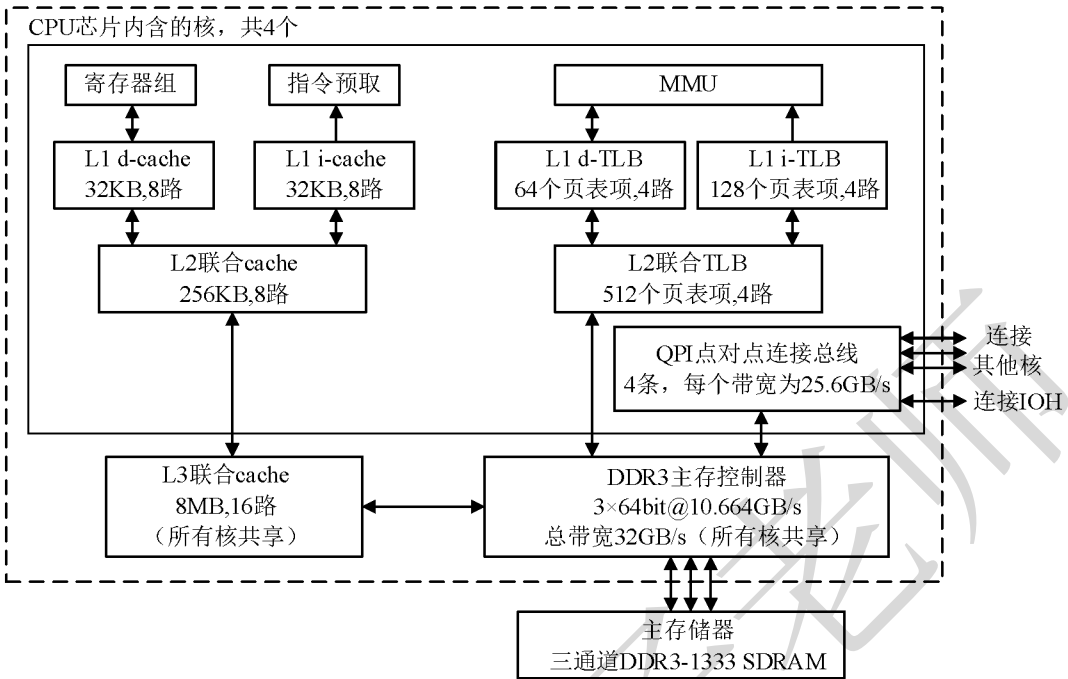


图 8.4 Intel Core i7 中各类存储器以及核之间的互连

3.I/O 总线

I/O 总线用于为系统中的各种 I/O 设备提供输入/输出通路，物理上通常是主板上的一些 I/O 扩展槽。早期的第一代 I/O 总线有 XT、ISA、EISA、VESA,这些 I/O 总线早已被淘汰；第二代 I/O 总线包括 PCI、AGP、PCI-X；第三代 I/O 总线是 PCI-Express。

与前两代 I/O 总线采用并行传输的同步总线不同，PCI-Express 采用串行传输方式。两个 PCI-Express 设备之间以一个链路(link)相连，每个链路可包含多条通路(lane)，可能的通路数为 1、2、4、8、16 或 32，PCI-Express×n 表示具有 n 个通路的 PCI-Express 链路。

每条通路由发送和接收数据线构成，在发送和接收两个方向上都各有两条差分信号线，可同时发送和接收数据。在发送和接收过程中，每个数据字节实际上被转换成了 10 位信息后才被传输，以保证所有位都含有信号电平的跳变。这是因为在链路上没有专门的时钟信号，接收器使用锁相环(PLL)从进入的位流 0-1 和 1-0 跳变中恢复时钟。

PCI-Express 1.0 规范支持通路中每个方向的发送或接收速率为 2.5Gb/s。因此，PCI-Express 1.0 规范总线的总带宽计算公式(单位为 GB/s)为

$$\text{PCI-Express 1.0 规范总线带宽} = 2.5\text{Gb/s} \times 2 \times \text{通路数} / 10$$

根据上述公式可知，在 PCI-Express1.0 规范下，PCI-Express×1 的总带宽为 0.5GB/s；PCI-Express×2 的总带宽为 1GB/s；PCI-Express×16 的总带宽为 8GB/s。

【图领 1800 题举例】

4. 【2019 年统考真题】某计算机采用 3 通道存储器总线，配套的内存条型号为 DDR3-1333，即内存条所接插的存储器总线的工作频率为 1333MHz，总线宽度为 64 位，则存储器总线的总带宽大约是（ ）。

- A. 10.66 GB/s B. 32 GB/s C. 64 GB/s D. 96 GB/s

5. 【2020 年统考真题】QPI 总线是一种点对点全双工同步串行总线，总线上的设备可同时接收和发送信息，每个方向可同时传输 20 位信息（16 位数据+4 位校验位），每个 QPI 数据包有 80 位信息，分 2 个时钟周期传送，每个时钟周期传递 2 次。因此，QPI 总线带宽为：每秒传送次数×2B×2。若 QPI 时钟频率为 2.4GHz，则总线带宽为（ ）。

- A. 4.8 GB/s B. 9.6 GB/s C. 19.2 GB/s D. 38.4 GB/s

6. 【2025 年统考真题】某处理器总线采用同步，并行传输方式，每个总线时钟周期传送 4 次数据（quadpumped 技术），若该总线的工作频率为 1333MHz（实际单位是 MT/S，表示每秒传送 1333M/次），总线宽度为 64 位，则总线带宽约为（ ）。

- A. 10.66 GB/s B. 42.66 GB/s C. 85.31 GB/s D. 341.25 GB/s

7. PCI-Express 采用串行传输方式。两个 PCI-Express 设备之间以一个链路相连，每个链路可包含多条通路，可能的通路数为 1、2、4、8、16 或 32，PCI-Express×n 表示具有 n 个通路的 PCI-Express 链路。每条通路由发送和接收数据线构成，在发送和接收两个方向上都各有两条差分信号线可同时发送和接收数据。在发送和接收过程中，每个数据字节实际上被转换成了 10 位信息后才被传输，以保证所有位都含有信号电平的跳变。对于 PCIeExpress1.0，其支持通路中每个方向的发送或接收速率为 2.5Gb/s。故 PCIeExpress1.0 规范总线的总带宽计算公式为 $2.5\text{Gb/s} \times 2 \times \text{通路数} / 10$ ，则 PCI-Express×32 的总线带宽为（ ）。

- A. 16 Gb/s B. 16 GB/s C. 32 GB/s D. 32 Gb/s

6.2 总线事务

对比王道 6.2.1 总线事务

（一）未解释突发传送和非突发传送的相关对比

【图领强化讲义】

【考察方式】选择题

【深入剖析】

总线的事务类型中非突发传输方式和突发传输方式是重点学习内容，下面用一个表格对比着来学习这两种传输方式。

维度	非突发传输	突发传输	关键结论
寻址次数	每传输 1 个数据单元需 1 次寻址	传输 n 个数据单元 仅需 1 次寻址	突发传输减少 $(n-1)$ 次寻址时间
时间消耗（假设）	单次：寻址 $1T$ + 传输 $1T$ + 准备数据的时间 $3T$ ，则 n 次需要 $n \times 5T$	总：寻址 $1T + n \times$ 传 输 $1T +$ 准备数据 $3T$ $\rightarrow$ 总 $4T + nT$	/
典型应用	I/O 端口读写（单次少量 数据）	内存加载、显卡帧缓 冲更新（连续数据）	传输数据量越大，突 发优势越明显
考试计算点	总时间 = 数据个数 $\times$ (寻址时间 + 传输时间 + 准备数据的数据)	总时间 = 寻址时间 + 准备第一个数据的 时间 数据个数 $\times$ 传输时间	需明确“寻址时间” 和“传输时间”的时 钟周期数

【1800 题举例】

8.【图领★】【2023 年统考真题】某存储器总线宽度为 64 位，总线时钟频率为 1GHz，在总线上传输一个数据或地址需要一个时钟周期，不支持突发传送方式。若通过该总线连接 CPU 和主存，主存每次准备一个 64 位数据需要 6ns，主存块大小为 32B，则读取一个主存块所需的时间是（ ）。

- A. 8 ns B. 11 ns C. 26 ns D. 32 ns

11.【图领★】【2012 年统考真题】某同步总线的时钟频率为 100MHz，宽度为 32 位，地址/数据线复用，每传输一个地址或数据占用一个时钟周期。若该总线支持突发（猝发）传输方式，则一次“主存写”总线事务传输 128 位数据所需要的时间至少是（ ）。

- A. 20 ns B. 40 ns C. 50 ns D. 80 ns

12.某 32 位同步总线时钟频率 200MHz，采用四体交叉存储方式，每个体的存储字长为 32 位，存取周期为 20ns，支持突发传送总线事务，每次读突发传送总线事务的过程包

括传送首地址和命令、存储器准备数据、传送数据。每次突发传送 16 B，传送地址 或 32 位数据需一个总线时钟周期，则传输 32B 数据完成的时间为()，数据传输速率是()。

A. 85 ns, 356 MB/s

B. 90 ns, 356 MB/s

C. 85 ns, 376 MB/s

D. 90 ns, 376 MB/s

第七章

7.2 I/O 接口

（一）未解释 I/O 指令的相关定义和类型

在底层 I/O 软件中，可以将控制命令送到控制寄存器来启动外设工作；可以读取状态寄存器来了解外设和设备控制器的状态；可以通过直接访问数据缓冲寄存器来进行数据的输入和输出。当然，这些对数据缓冲寄存器、控制/状态寄存器的访问操作是通过相应的指令来完成的，通常把这类指令称为 I/O 指令。因为这些 I/O 指令只能在操作系统内核的底层 I/O 软件中使用，因而它们是一种特权指令。

例如，IA-32 中提供了 4 条专门的 I/O 指令：`in`、`ins`、`out` 和 `outs`，其中的 `in` 和 `ins` 指令用于将 I/O 接口中某个寄存器的内容取到 CPU 内的通用寄存器中；`out` 和 `outs` 用于将通用寄存器的内容输出到 I/O 接口的某个寄存器中。

52. 【图领★】【2017 年统考真题】I/O 指令实现的数据传送通常发生在（ ）。

- A. I/O 设备和 I/O 端口之间
- B. 通用寄存器和 I/O 设备之间
- C. I/O 端口和 I/O 端口之间
- D. 通用寄存器和 I/O 端口之间

53. 下列关于 I/O 指令的说法中，错误的是（ ）。

- A. 独立编址方式必须使用专用 I/O 指令实现 CPU 与 I/O 设备的数据交互
- B. 统一编址方式，可通过存储器访问指令代替 I/O 指令，简化指令系统
- C. I/O 指令的实现方式包括直接寻址、间接寻址和立即数寻址等
- D. I/O 指令只能实现数据输入/输出操作，无法控制 I/O 设备的工作状态

7.3 I/O 方式

对比王道 7.3.1 程序查询方式。

（一）未解释清楚程序查询方式的流程和适用

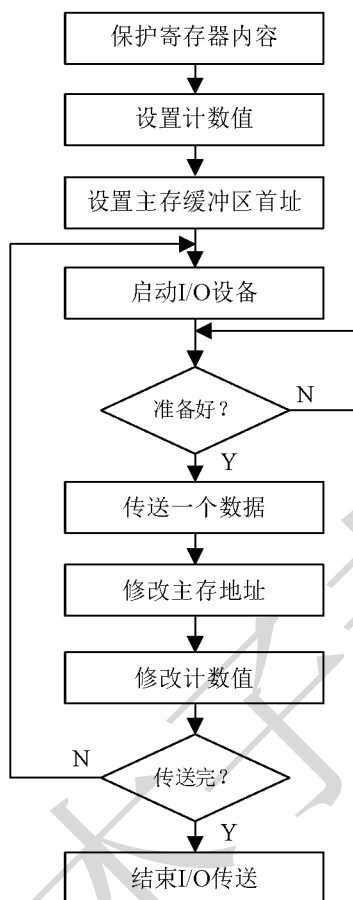


图 5.34 程序查询方式的程序流程

①由于这种方式传送数据时要占用 CPU 中的寄存器，故首先需将寄存器原内容保护起来(若该寄存器中存有有用信息)。

②由于传送往往是一批数据，因此需先设置 I/O 设备与主机交换数据的计数值。

③设置欲传送数据在主存缓冲区的首地址。

④CPU 启动 I/O 设备。

⑤将 I/O 接口中的设备状态标志取至 CPU 并测试 I/O 设备是否准备就绪。如果未准备就绪，则等待，直到准备就绪为止。当准备就绪时，接着可实现传送。对输入而言，准备就绪意味着接口电路中的数据缓冲寄存器已装满欲传送的数据，称为输入缓冲满，CPU 即可取走数据；对输出而言，准备就绪意味着接口电路中的数据已被设备取走，故称为输出缓冲空，这样 CPU 可再次将数据送到接口，设备可再次从接口接收数据。

⑥CPU 执行 I/O 指令，或从 I/O 接口的数据缓冲寄存器中读出一个数据，或把一个数

据写入 I/O 接口中的数据缓冲寄存器内，同时将接口中的状态标志复位。

⑦修改主存地址。

⑧修改计数值，若原设置计数值为原码，则依次减 1;若原设置计数值为负数的补码，则依次加 1(有关原码、补码的概念可参阅 6.1 节)。

⑨判断计数值。若计数值不为 0，表示一批数据尚未传送完，重新启动外设继续传送;若计数值为 0，则表示一批数据已传送完毕。

⑩结束 I/O 传送，继续执行现行程序。

个人补充：

王道教材中所描述的查询方式是适用于启动一次，连续传送的多次数数据的特殊外设。这里描述的是教材中普遍采用的查询方式，应当对这部分进行区分。

对比王道 7.3.2 程序中断方式。

(一) 未解释中断控制 I/O 的工作过程

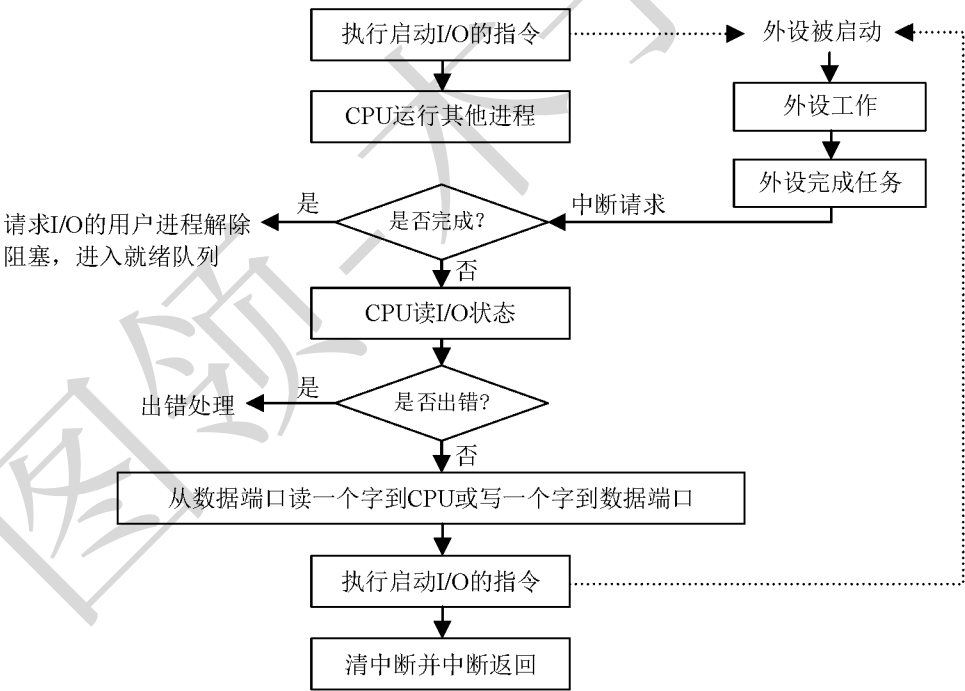


图 8.14 中断控制 I/O 过程

①启动外设与 CPU 并行执行。

CPU 执行启动 I/O 的指令，向目标外设发送启动命令，外设被激活并开始独立执行 I/O

操作。

优点：CPU 无需等待外设完成，立即切换到其他进程继续执行，实现 CPU 与外设的并行工作。

②外设完成任务并发送中断请求。

外设按照指令自主完成 I/O 操作（如数据准备就绪、数据输出完成）。当外设完成本次任务后，通过中断控制器向 CPU 发送中断请求信号，通知 CPU 自己已经准备好进行下一步交互。

③CPU 响应中断并判断任务完成状态。

CPU 在每条指令执行周期的末尾检查中断请求，检测到该外设的中断后，暂停当前进程、保护现场，转去执行对应的中断服务程序，首先判断整个 I/O 传输任务是否全部完成：

（1）是（任务全部完成）：将之前因等待该 I/O 操作而阻塞的用户进程唤醒，使其从阻塞状态转为就绪状态，进入进程就绪队列等待调度。

（2）否（仅完成单个数据单元传输，整体任务未结束）：进入下一步状态检查。

④I/O 状态与错误检查。

CPU 读取外设的状态寄存器，检查本次 I/O 操作是否出现异常：

（1）是（检测到错误）：转入出错处理程序，执行重试、报错等异常处理逻辑。

（2）否（操作正常）：进入数据传输阶段。

⑤单数据单元传输。

CPU 与外设进行一次数据交换：

（1）输入操作：从外设的数据端口读取一个字的数据到 CPU 内部寄存器。

（2）输出操作：将 CPU 内部寄存器中的一个字的数据写入外设的数据端口。

⑥再次启动外设与中断返回。

完成本次数据单元传输后，CPU 再次执行启动 I/O 的指令，启动外设进行下一个数据单元的传输。随后执行清中断并中断返回操作：清除本次中断请求，恢复被中断进程的现场，回到被中断的进程继续执行。

⑦循环与终止。

上述步骤 3~6 会循环执行，直到整个 I/O 传输任务全部完成，此时在“是否完成？”的判断中走“是”分支，唤醒阻塞的用户进程，本次中断控制 I/O 过程结束。

(二)、未给出中断响应判优的优先级的具体情况。

【图领强化讲义】

【图领提示】

中断优先级通常包括处理优先级和响应优先级。

响应优先级指当多个中断源同时发出中断请求时，CPU 确定对它们响应的先后顺序，主要由硬件电路或中断控制器根据预先设定的规则来确定，一般不可动态改变。

CPU 响应中断后，开始执行中断服务程序时，会根据中断源的处理优先级来确定处理顺序，其可以利用中断屏蔽技术来动态调整，一般用于实现中断嵌套。

通常根据中断事件的重要性和迫切性来划分中断响应优先级，一般的划分规律如下

(1) 不可屏蔽中断>内部异常>可屏蔽中断。

(2) 内部异常中硬件终止属于最高级，其次是指令异常或自陷等程序故障。

(3) DMA 中断请求优先于 I/O 设备传送的中断请求。

(4) 在 I/O 传送类中断请求中，高速设备优先于低速设备，输入设备优先于输出设备，实时控制设备优先于普通设备。

【1800 题举例】

28.CPU 对于以下事件的响应顺序是 ()。

I. INTn 指令中断 II. DMA 请求 III. 电源掉电 IV. 键盘输入中断

A. I→III→II→IV

B. II→III→I→IV

C. III→II→I→IV

D. III→II→IV→I

对于I，INTn 指令是一种自陷指令。执行 INTn 指令会引起内中断，该中断属于自陷异常。高于可屏蔽中断(INTR)。

对于II，DMA 请求的优先级高于中断请求的优先级，高于可屏蔽中断(INTR)。

对于III，电源掉电属于不可屏蔽中断，若发现有掉电的可能，立即向 CPU 发出一个最高优先级的中断，CPU 立即保存一些重要的数据，保护一些特定的硬件，同时向外设发出一些信息等。该种类型的中断优先级极高。

对于IV，键盘输入中断属于可屏蔽的中断(INTR)，在 4 个选项中优先级最低。故正确

答案为 C。

个人补充：

对于嵌套的多重中断系统中，关中断在中断响应阶段是由硬件（中断隐指令）自动完成的，而在恢复现场和中断屏蔽之前的关中断是由关中断指令完成的。

对比王道 7.3.3 DMA 方式

（一）未说明 DMA 与存储系统的访存情况

3.DMA 与存储系统

当采用 DMA 方式进行 I/O 时，存储系统和 CPU 之间的关系会变得更复杂。没有 DMA 控制器时，所有对主存的访问都来自 CPU，通过 MMU 中的地址转换和 cache 进行访存。有了 DMA 控制器后，系统中就有了另一个访问主存的路径，它不通过地址转换机制和 cache 层次。这样，在虚拟存储器和 cache 系统中就会产生一些问题。这些问题的解决通常要结合硬件和软件两方面的技术支持。

在采用虚拟存储器的系统中，同时有物理地址和虚拟地址，那么 DMA 是以虚拟地址还是以物理地址工作呢？很明显，若用虚拟地址，则在 DMA 接口中应有一个小的类似页表的地址映射表，用于将虚拟地址转换为物理地址；若用物理地址，则每次 DMA 传送不能跨页。如果一个 I/O 请求跨页，那么一次请求的一个数据块在送到主存时，就可能不在主存的一个连续的存储区中，因为每个虚页可以映射到主存的任意一个实页框中，所以多个连续的虚页不可能正好对应连续的实页框。因此，如果 DMA 采用物理地址，就必须限制所有的 DMA 传送都必须在一个页面之内进行，这种情况下，由操作系统把一次传送分解成多次小数据量传送，以保证每次只限定在一个物理页面内进行。

采用 cache 的系统中，一个数据项可能会产生两个副本，一个在 cache 中，一个在主存储器中，因此，具有 DMA 控制器的 cache 系统也会产生问题。DMA 控制器直接向主存储器发出访存请求而不通过 cache，这时，DMA 控制器看到的一个主存单元的值与 CPU 看到的 cache 中的副本可能不同。考虑从磁盘中读一个数据，DMA 直接将其送到主存，如果有些被 DMA 写过的单元在 cache 中，那么以后 CPU 读取这些单元的时候，就会得到一个老的值。类似地，如果 cache 采用写回(write back)策略，当一个新的值在 cache 中写入

时，这个值并未被马上写回主存，而此时若 DMA 直接从主存读，那么读的值可能是老的值。这个问题称为过时数据问题或 I/O 一致性问题，其解决方法有以下 3 种。

第一种解决方式是让 I/O 活动通过 cache 进行，这样就保证了在 I/O 读时能读到最新的数据，而 I/O 写时能更新 cache 中的任何数据。当然，将所有 I/O 都通过 cache，其代价是非常大的。因为 I/O 数据通常很少马上要用到，如果这样的数据把 CPU 正在使用的有用数据替换出去，那么就会影响 cache 的命中率，给 CPU 的性能带来很多负面影响。

第二种解决方式是让操作系统在 I/O 读时有选择地使某些 cache 块无效，而在 I/O 写时迫使 cache 进行一次写回操作，这种操作经常被称为 cache 刷新，这种方式需要少量硬件支持。因为大部分 cache 刷新操作仅发生在 DMA 数据块访问时，而 DMA 访问又不经常发生，所以，如果软件能方便而有效地实现这种方法，则性价比更高。

第三种解决方式是通过一个硬件机制来选择被刷新或使无效的 cache 项，这种用硬件来保证 cache 一致性的方式大多被用在多处理器系统中。